



Документация

ShredShare

СТУДЕНТИ: Теодор Пламенов Петров

ФАКУЛТЕТЕНИ НОМЕРА: 121223090

СПЕЦИАЛНОСТ: КОМПЮТЪРНО И СОФТУЕРНО ИНЖЕНЕРСТВО

ГРУПА: 43 гр.

ДИСЦИПЛИНА: ПРОГРАМИРАНЕ ЗА МОБИЛНИ УСТРОЙСТВА

ТУ - СОФИЯ 2025

Съдържание

1. Увод.....	4
2. Анализ на съществуващи разработки	4
3. Проектиране	5
3.1 Потребители на системата и техните роли	6
3.2 Модел на приложението	6
3.3 Данни използвани в приложението	7
3.4 Анализ на входните данни от потребителя.....	8
3.5 Изграждане на данните в структури	9
3.6 Описание на основните бизнес процеси.....	11
3.7 Достъп до функционалностите от страна на потребителя.....	14
4. Реализация	15
4.1 Бази данни.....	15
4.2 Backend (Spring Boot).....	18
1. Entry point и конфигурация	18
1. Структура.....	22
2. Entity класове.....	24
3. Repository класове	26
4. DTO класовете.....	28
5. Service класове	29
6. Controller класове	31
4.3 Андроид реализация	33
1. Структура	33
2. Gradle конфигурация и AndroidManifest.....	35
3. MainActivity	36
4. Navigation.....	37
5. Data	41
7. ViewModel.....	45
8. UI	47
9. Validation класове	50
5. Потребителско ръководство	51
1. Начален екран.....	51

2.	Екран за избор на роля	52
3.	Екран за Регистрация.....	53
4.	При избор на регистрация като потребител	54
5.	Екран „Гардероби в курорта“	55
6.	Екран „Артикули в гардероба“	56
7.	Екран „Детайли за артикул“	58
8.	Екран „Количка“	58
9.	Екран „Моите резервации“	59
10.	Екран „Профил“	60
11.	При избор на регистрация като собственик	61
12.	Екран „Табло на собственика“	62
13.	Екран „Управление на артикули“	63
14.	Екран „Добавяне на артикул“	64
15.	Екран „Управление на поръчки“	65
16.	Екран за администратор	66
17.	Екран за забравена парола.....	67
6.	Заключение	67

1. Увод

През последните години се наблюдава значително нарастване на интереса към зимните спортове като ски и сноуборд, както в световен мащаб, така и в България. Курорти като Банско, Пампорово и Боровец привличат ежегодно хиляди туристи, което води до повишено търсене на специализирана спортна екипировка.

Закупуването на качествени ски или сноуборд, заедно със съпътстващите аксесоари (обувки, каски, очила и др.), представлява значителна инвестиция. Освен финансовия аспект, съществуват и редица други неудобства, свързани с транспортиране, съхранение и поддръжка на екипировката. Поради тези причини, услугите за отдаване под наем на спортна екипировка се утвърждават като предпочитано решение за голяма част от потребителите.

Въпреки наличието на множество физически магазини и пунктове за отдаване под наем, процесът по намиране, сравняване и резервиране на подходяща екипировка често е затруднен. Основните проблеми включват липса на централизирана информация за наличностите, ограничена прозрачност относно цените, както и отсъствие на удобни дигитални инструменти за предварителна резервация. В повечето случаи потребителите са принудени да разчитат на място, което води до загуба на време, несигурност и потенциално по-неблагоприятни условия.

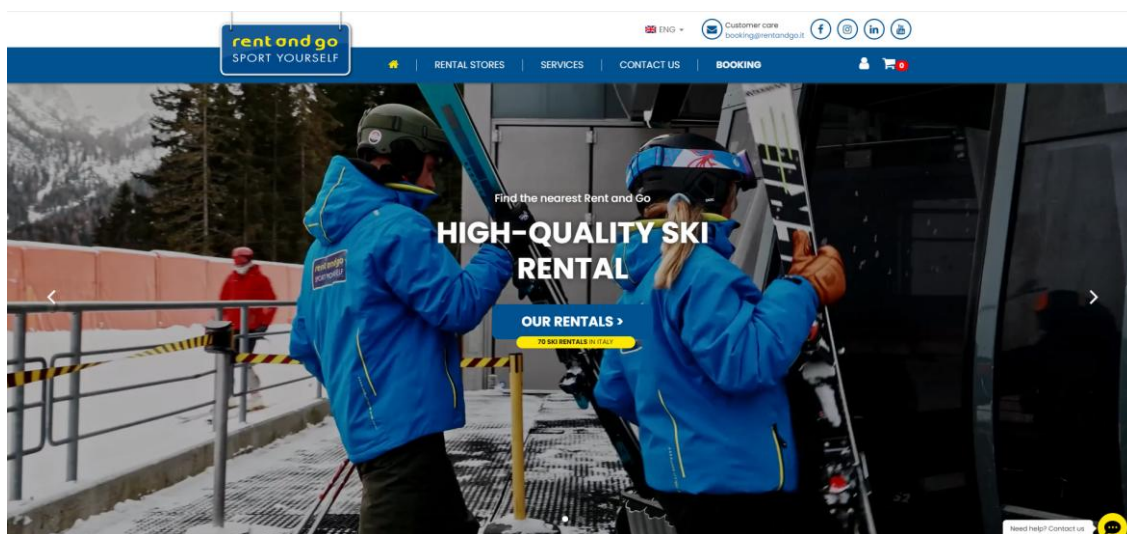
От друга страна, собствениците на подобни услуги също се сблъскват с предизвикателства, свързани с управлението на инвентара, проследяването на наличностите и комуникацията с клиентите. Липсата на интегрирана система за управление често води до неефективност, грешки при резервации и пропуснати възможности за оптимизиране на бизнеса. Така възниква необходимостта от създаване на модерно софтуерно решение, което да обедини всички участници в процеса – клиенти, собственици и администратори – в една обща платформа. **ShredShare**, представлява мобилно приложение, разработено с цел да улесни и оптимизира процеса по наемане на ски и сноуборд екипировка.

Основната идея на приложението е да предостави централизирана система, чрез която потребителите могат лесно да разглеждат наличната екипировка, да филтрират резултатите според своите нужди, да извършват резервации и да управляват своите заявки. В същото време, собствениците получават възможност да добавят и управляват своя инвентар, да следят поръчките и да оптимизират дейността си. Администраторите от своя страна осигуряват контрол върху системата, одобряват заявки и гарантират коректното функциониране на платформата.

2. Анализ на съществуващи разработки

Една от най-разпознаваемите платформи в тази област е Rent and Go, която предлага възможност за онлайн резервация на ски и сноуборд екипировка чрез мобилно приложение и уеб интерфейс. Потребителите могат да избират между различни курорти и магазини, като резервират екипировка предварително, което значително улеснява процеса при пристигане на място. Платформата предлага сравнително удобен потребителски интерфейс и бърз процес на резервация. Въпреки това, тя има редица ограничения. Основен недостатък е липсата на разширена персонализация и ограничените възможности за филтриране на оборудването. Освен това, системата не поддържа гъвкава ролева структура, при която различни типове потребители (например собственици и администратори) да имат специфични права за управление. По отношение на географското разпространение, Rent and Go

се използва основно в държави от Централна и Западна Европа, като Австрия, Италия, Германия и Франция, но има ограничено присъствие в Източна Европа, включително България.



Фиг. 1 Rent and Go website

Друго решение, което намира широко приложение в управлението на наемни услуги, е Boobookable. Това е облачна система, предназначена основно за бизнеси, които отдават под наем различни видове оборудване, включително ски и сноуборд екипировка. Платформата предоставя мощни инструменти за управление на инвентар, проследяване на наличности, обработка на резервации и автоматизация на бизнес процесите. Нейно основно предимство е високото ниво на контрол и организация, което предлага на собствениците на магазини. Въпреки това, Boobookable не е ориентирана към крайния потребител, а към администриране на бизнеси. Липсва marketplace функционалност, при която различни доставчици да се обединяват в една платформа, както и мобилно приложение, насочено към крайни клиенти. Географски, системата е достъпна глобално, но се използва основно в развити пазари като САЩ, Великобритания, Нидерландия и други страни от Западна Европа, което я прави по-малко популярна в по-малки и локални пазари.

В резултат на направения анализ може да се заключи, че съществуващите решения, макар и функционални, имат съществени ограничения. Повечето от тях са насочени към управление на бизнес процеси, а не към създаване на удобна платформа за крайни потребители. Освен това, липсва единна система, която да обединява различни доставчици на екипировка в централизирана среда. Географските ограничения също играят важна роля, тъй като много от разглежданите решения не са широко достъпни или популярни в България и региона. Тези недостатъци обосновават необходимостта от разработването на ShredShare – приложение, което комбинира удобен потребителски интерфейс, marketplace функционалност, управление на инвентар и гъвкава ролева система, като същевременно е насочено към локалния пазар и нуждите на потребителите в региона.

3. Проектиране

Проектирането на ShredShare е насочено към създаването на модерна мобилна система за отдаване под наем на ски и сноуборд екипировка, която да съчетава удобство за крайния потребител, лесно управление за собствениците на гардероби и ефективен контрол от страна на администратора. За да се постигне това, е необходимо да бъдат разглеждани няколко основни аспекта: кой ще използва продукта, какви данни ще се обработват и съхраняват, както и по какъв начин потребителят ще достъпва различните функционалности.

3.1 Потребители на системата и техните роли

При проектирането на системата първата важна стъпка е определянето на основните групи потребители, които ще работят с приложението. Това е необходимо, тъй като различните типове потребители имат различни цели, права за достъп и нужди, а оттук произтичат и разликите във функционалността, която приложението трябва да им предостави.

В приложението **ShredShare** са дефинирани три основни типа потребители: **клиент**, **собственик** и **администратор**.

Първият и най-важен тип потребител е **клиентът**. Това е крайният потребител на системата, който използва приложението, за да търси, разглежда и наема ски или сноуборд екипировка. При проектирането на функционалностите за този тип потребител е важно да се осигури бърз и интуитивен достъп до каталога с артикули, възможност за избор на курорт, филтриране по категории, размери и тип екипировка, както и създаване на поръчка или резервация за конкретен период. От гледна точка на потребителското изживяване, клиентът трябва да може лесно да се ориентира в системата, да разбира какви артикули са налични и какви действия може да извърши без излишна сложност.

Вторият тип потребител е **собственикът**. Това е лице или бизнес, който предлага екипировка под наем чрез системата. За него приложението трябва да предоставя възможности за създаване и управление на профил, регистриране на гардероб или обект, добавяне на нови артикули, редактиране на вече въведена екипировка, качване на изображения, определяне на цена, описание, размери и наличности. Освен това собственикът трябва да може да проследява заявките, направени от клиентите, и да управлява информацията за своя инвентар. Този тип потребител има нужда от по-административен интерфейс, насочен към управление на съдържание и бизнес процеси.

Третият тип потребител е **администраторът**. Той отговаря за общия контрол върху системата и има задача да гарантира, че приложението функционира коректно. Неговите основни права включват преглед и одобрение на заявки, управление на потребители, следене на състоянието на гардеробите и контрол върху данните в системата. Администраторът не използва приложението по същия начин като клиента, а има надзорна и организационна функция. Поради това интерфейсът за администратора трябва да бъде ориентиран към преглед, моделиране и вземане на управленски решения.

От това разпределение става ясно, че приложението трябва да бъде ролево ориентирано, тоест поведението на системата, показваните екрани и достъпните функции да зависят от ролята на вписания потребител. Това решение е важно, защото осигурява по-добра сигурност, по-логична навигация и по-ясно разделяне на отговорностите.

3.2 Модел на приложението

Приложението **ShredShare** е проектирано като система с **клиент–сървърна архитектура**, при която мобилното приложение изпълнява ролята на клиентска част, а бекенд сървърът обработва бизнес логиката и комуникацията с базата данни.

От едната страна се намира **Android приложението**, разработено с помощта на **Jetpack Compose**, което предоставя визуалния интерфейс и приема действията на потребителя. То отговаря за визуализацията на списъци с артикули, екраните за регистрация и вход, профилните страници, количката, формите за добавяне и редактиране на екипировка, както и навигацията между отделните екрани.

От другата страна се намира **Spring Boot бекендът**, който изпълнява ролята на централен модул за обработка на данни и бизнес процеси. Той получава заявки от мобилното приложение, валидира входните данни, изпълнява логиката по регистрация, автентикация, управление на артикули, обработка на поръчки и заявки за одобрение, след което връща отговор към клиента. В бекенда са организирани контролери, сервизни класове и repository слой за връзка с базата данни.

В основата на системата стои **MySQL база данни**, която съхранява всички ключови данни – потребители, курорти, гардероби, артикули, колички, поръчки, заявки и други свързани обекти. По този начин се осигурява централизация на информацията, последователност на бизнес данните и възможност за проследяване на състоянието на системата.

Такъв тип архитектура е избрана, защото позволява добро разделяне между потребителски интерфейс, бизнес логика и слой за съхранение на данни. Това улеснява поддръжката, бъдещото разширяване на системата и добавянето на нови функционалности.

3.3 Данни използвани в приложението

За да функционира системата коректно, е необходимо да бъдат дефинирани основните информационни обекти и връзките между тях.

Основният обект в системата е **потребителят**. За него се съхраняват данни като собствено име, фамилия, имейл адрес, парола, телефонен номер и роля. Имейл адресът служи като уникален идентификатор при вход, а ролята определя достъпните функционалности в приложението.

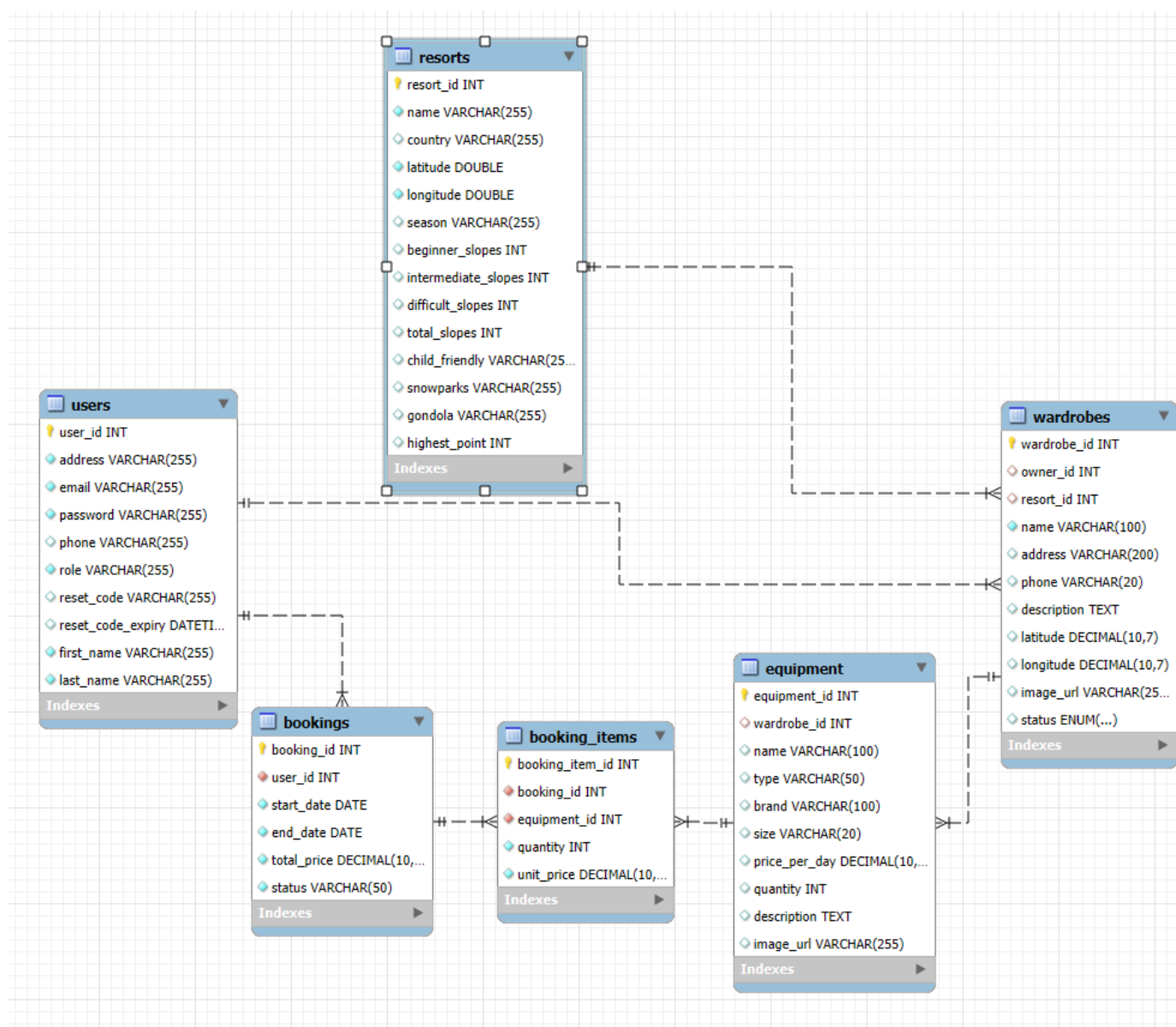
Следващ важен обект е **курортът**. Той е необходим, защото ShredShare е насочено към зимни спортове и наличната екипировка трябва да бъде свързана с конкретна локация. За курорта се съхраняват данни като име, местоположение, кратко описание и др. Използван е dataset с всички налични курорти в Европа, както и известни курорти в световен мащаб.

Друг ключов обект е **гардеробът** или обектът за отдаване под наем. Той принадлежи на конкретен собственик и е свързан с определен курорт. За него се пазят данни като име, адрес, телефон, описание, изображение и статус. Статусът може да се използва например за отбелязване дали гардеробът е в изчакване за одобрение, одобрен или отказан от администратора.

Сърцевината на приложението е обектът **екипировка**. Това са конкретните артикули, които клиентът вижда и може да наеме. За всеки артикул се съхраняват име, категория, размер, марка, цена за ден, количество, описание, изображение и връзка към гардероба, към който принадлежи. Този обект е от решаващо значение, тъй като именно чрез него се реализира бизнес стойността на приложението.

За да може потребителят да извършва реална поръчка, в системата е необходим обект от тип **резервация**. В него се пазят данни като клиент, избрани артикули, период на наемане, статус на заявката и обща цена. Към всяка поръчка са свързани отделни позиции, които описват кой артикул е добавен, в какво количество и на каква цена.

Проектирането на тези данни има значение не само за съхранението им в база данни, но и за начина, по който те се моделират в програмния код чрез класове, DTO обекти, entity класове и repository интерфейси. Всеки обект от реалния свят има своя програмна репрезентация, като например User, Wardrobe, Equipment, Resort и други.



Фиг. 2 ER диаграма на базата данни

3.4 Анализ на входните данни от потребителя

В ShredShare съществуват няколко основни групи входни данни.

Първата и най-важна група са данните за потребителите. При регистрация всеки потребител въвежда собствено име, фамилия, имейл, адрес, парола и телефонен номер, както и избира роля в системата – клиент, собственик или администратор. Имейлът служи като уникален идентификатор и не трябва да се дублира. Паролата трябва да отговаря на минимални изисквания за сигурност, като дължина и сложност, а телефонният номер следва да бъде във валиден формат. Освен тези основни данни, системата поддържа и механизъм за възстановяване на парола чрез генериране на код за нулиране и срок на неговата валидност. Това изисква допълнителни проверки и защита срещу злоупотреби.

Следващата група данни е свързана с курортите. Тези данни включват име на курорта, държава, географски координати, сезонност и различни характеристики на пистите, като брой и тип (начинаещи, средни, трудни). Макар потребителите да не въвеждат тези данни директно, те играят ключова роля при избора на екипировка и филтрирането на резултатите в приложението.

Третата група входни данни е свързана с гардеробите (wardrobes), които се създават от собствениците. При добавяне на нов гардероб потребителят въвежда име на обекта, адрес, телефон, описание, както и географски координати и изображение. Освен това гардеробът трябва да бъде асоцииран с конкретен курорт и собственик. Важно е да се осигури валидиране на тези данни, като например да не се допуска празно име, невалиден телефон или липсваща връзка с курорт. Системата използва и поле за статус, чрез което се контролира дали гардеробът е одобрен от администратора.

Четвъртата група данни се отнася до екипировката (equipment), която представлява основния обект на приложение. При добавяне на нов артикул собственикът въвежда име, тип (например ски или сноуборд), марка, размер, цена на ден, количество, описание и изображение. Всеки артикул трябва да бъде свързан с конкретен гардероб. Валидацията тук е от съществено значение – цената трябва да бъде положителна стойност, количеството не трябва да бъде отрицателно, а задължителните полета трябва да бъдат попълнени. Освен това трябва да се гарантира, че въведените стойности са логически коректни и съвместими с останалите данни в системата.

Петата група входни данни е свързана с процеса на резервация или поръчка (bookings). Клиентът избира начална и крайна дата за наемане, като системата изчислява общата цена на база избраните артикули и техните цени. Важно ограничение тук е, че не трябва да се допуска избор на минали дати или крайна дата, която предхожда началната.

Свързана с това е и последната група данни – детайлите на поръчката (booking_items). Те включват конкретните артикули, които са част от дадена резервация, тяхното количество и единична цена. Тези данни се генерират автоматично на база действията на потребителя и не се въвеждат директно като отделна форма, но са резултат от избора на клиента и трябва да бъдат валидирани спрямо наличностите.

3.5 Изграждане на данните в структури

В приложението ShredShare този процес се реализира чрез използване на обектно-ориентиран подход, при който всеки основен елемент от предметната област се моделира като отделен клас със свои свойства, връзки и поведение. По този начин се постига ясна връзка между реалните бизнес обекти, таблиците в базата данни и логиката на приложението. Изграждането на данните в класове и обекти позволява системата да бъде по-лесна за разработка, поддръжка, разширяване и тестване.

На най-ниско ниво данните се съхраняват в релационната база данни под формата на таблици, но в програмния код те се представят чрез така наречените **entity класове**. Това са класове, които отразяват структурата на съответните таблици и съдържат полета, съответстващи на колоните в тях. По този начин всеки запис в дадена таблица може да бъде разглеждан като конкретен обект в паметта на приложението.

От гледна точка на структурата на приложението, entity класовете не са единствените, които се използват. В модерните софтуерни системи, включително и в ShredShare, е практика да се използват и **DTO класове** (Data Transfer Objects). Те представляват помощни структури, чиято основна роля е да пренасят данни между отделните слоеве на приложението – например между клиентската част и сървъра или между контролерите и сервизния слой. Използването на DTO класове е особено важно, защото не е добра практика директно да се изпращат entity обектите към клиентската част. Причината

е, че entity класовете често съдържат повече информация, отколкото е необходима за дадена операция, а понякога включват и чувствителни данни, които не трябва да бъдат достъпни отвън.

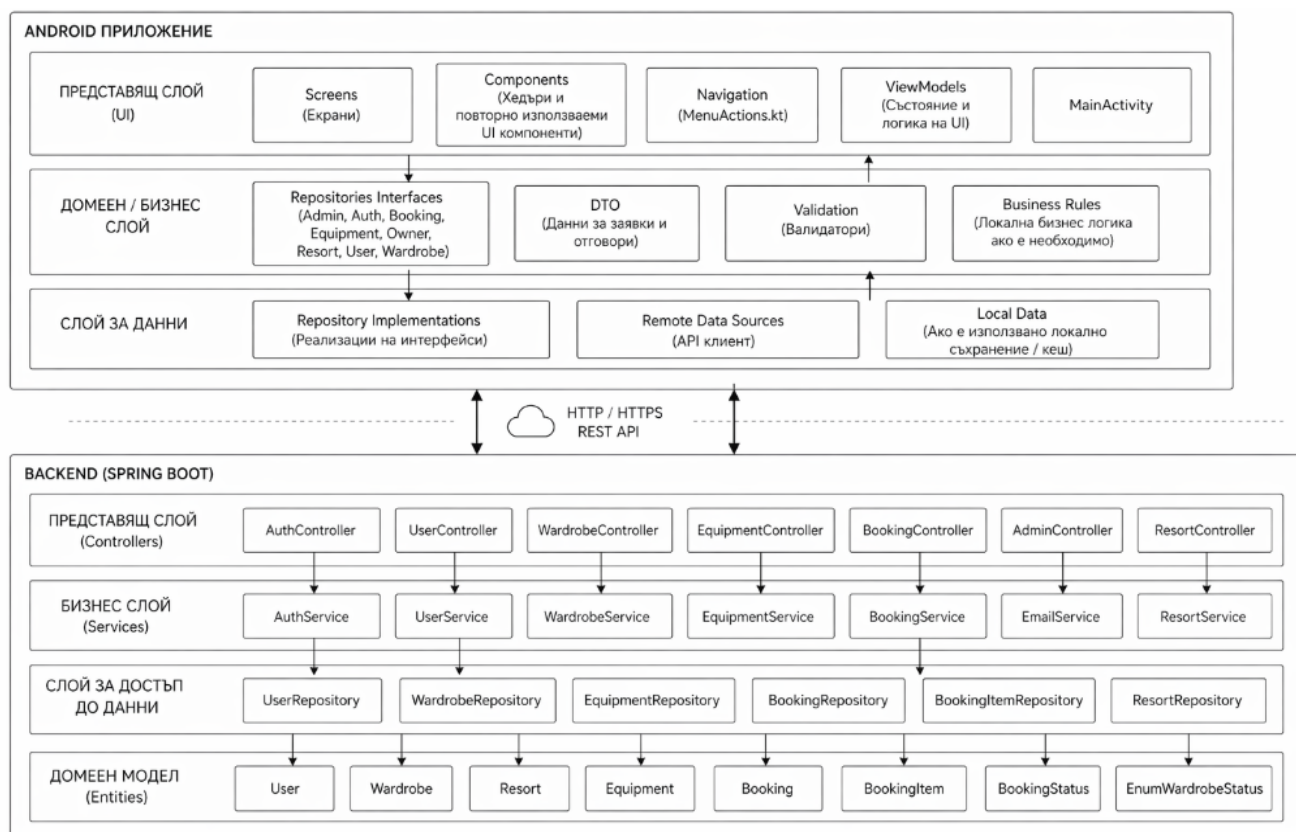
Например при регистрация на потребител може да се използва DTO клас от тип RegisterRequest, който съдържа само необходимите входни полета – име, фамилия, имейл, парола, телефон и роля. По аналогичен начин при вход в системата може да се използва LoginRequest, който съдържа само имейл и парола. При визуализация на артикулите към клиента е удачно да се използва DTO, което съдържа име, марка, размер, цена, количество, снимка и описание, без да се изпращат вътрешни технически данни, които не са необходими на интерфейса.

Освен entity и DTO класове, при изграждането на данните в приложението важна роля имат и **repository интерфейсите**, чрез които се осъществява достъпът до базата данни. Всеки основен клас има съответен repository слой, например UserRepository, WardrobeRepository, EquipmentRepository, BookingRepository и други. Тези интерфейси позволяват извличане, запис, редактиране и изтриване на обекти от базата данни. Те служат като връзка между логическия модел на приложението и физическото съхранение на данните.

Над repository слоя се намират **service класовете**, които реализират бизнес логиката върху тези обекти. В тях се извършват действия като проверка за съществуващ потребител по имейл, създаване на нов гардероб, валидиране на данни за екипировка, обработка на резервации, изчисляване на обща цена и промяна на статуси. Това означава, че обектите не съществуват само като структури за съхранение, а участват активно в логиката на системата.

На ниво клиентска част, тоест в Android приложението, тези данни се представят чрез допълнителни **модели за визуализация** и състояния на интерфейса. Това означава, че получените от сървъра данни често се преобразуват в по-удобен за екрана формат. Например списък от артикули може да бъде представен в UI модел, който съдържа само информацията, необходима за картата на продукта – име, снимка, цена, марка и наличност. Това е важно, защото интерфейсът и базата данни имат различни цели и не винаги трябва да използват едни и същи структури в чист вид.

Изграждането на данните в класове и обекти има още едно важно предимство – то позволява по-лесно разширяване на системата в бъдеще. Ако по-късно се добавят нови функционалности, като оценки, ревюта, онлайн плащания, известия или допълнителни категории екипировка, системата може да бъде надградена чрез добавяне на нови класове и връзки, без да се нарушава основната ѝ логика. Това прави архитектурата по-устойчива и мащабируема.



Фиг. 3 Архитектура на приложението

3.6 Описание на основните бизнес процеси

В ShredShare бизнес процесите са изградени около основната идея за отдаване под наем на ски и сноуборд екипировка и включват взаимодействие между трите основни роли – клиент, собственик и администратор.

Първият основен процес е **регистрация и автентикация на потребител**. Процесът започва с въвеждане на лични данни от потребителя – име, фамилия, имейл, парола, телефон и избор на роля. След изпращане на заявката към сървъра се извършва проверка за валидност на данните и за съществуващ потребител със същия имейл. При успешна регистрация се създава нов запис в таблицата users. При последващ вход в системата се извършва проверка на въведените идентификационни данни, като при успех се създава потребителска сесия и се определя достъпът до функционалностите според ролята.

Вторият съществен процес е **създаване и управление на гардероб от страна на собственика**. След успешен вход собственикът има възможност да създаде нов гардероб, като въведе данни като име, адрес, телефон, описание, координати и изображение, както и да избере курорт, към който принадлежи обектът. След създаване на гардероба той получава определен статус, който е „изчакващ одобрение“. Този статус определя дали гардеробът ще бъде видим за клиентите. Собственикът може също така да редактира или изтрива съществуващи гардероби, като тези действия се отразяват директно в базата данни.

Третият процес е **добавяне и управление на екипировка**. След като гардеробът е наличен в системата, собственикът може да добавя нови артикули към него. Това включва въвеждане на име на артикула, тип (ски, сноуборд и др.), марка, размер, цена на ден, количество, описание и изображение. След създаването на артикул, той се записва в таблицата equipment и става част от наличния инвентар на гардероба. Собственикът има възможност да актуализира информацията за екипировката, например при промяна на цена или наличност, както и да премахва артикули от системата.

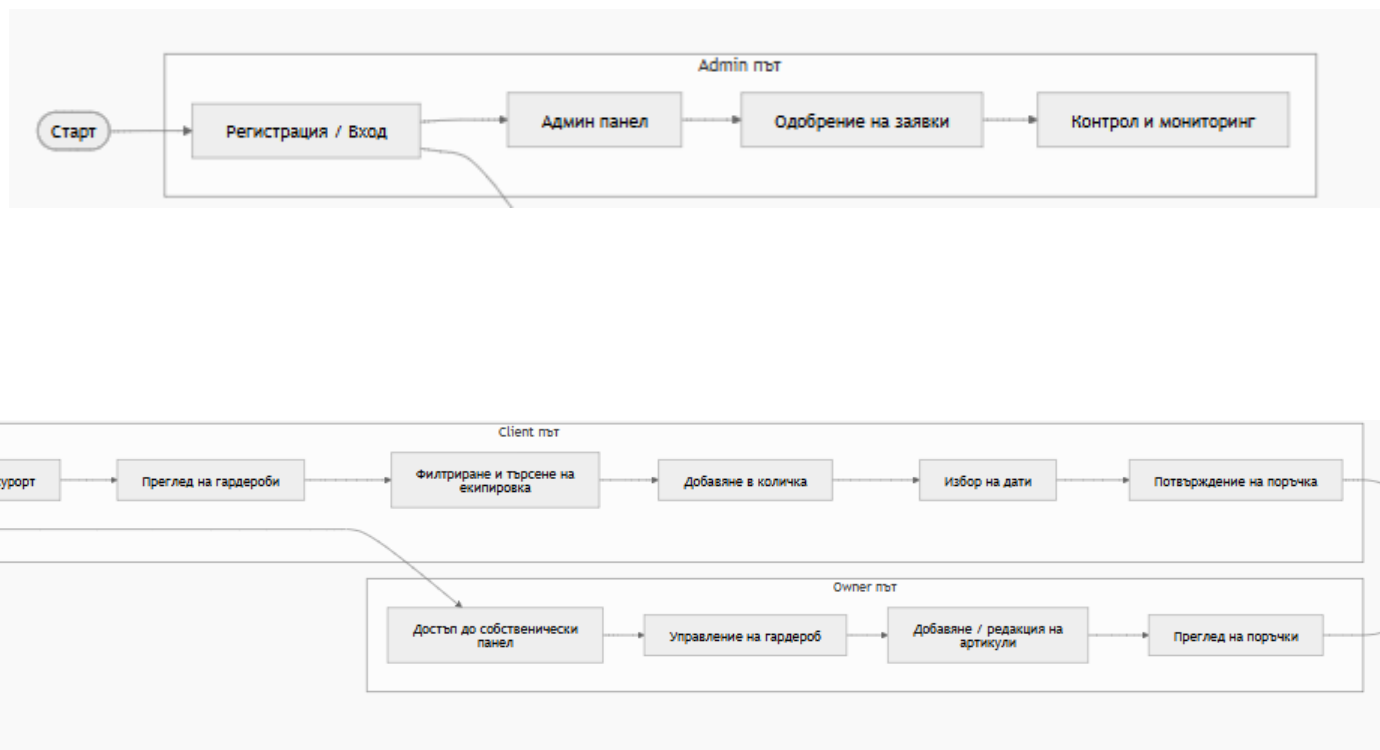
Четвъртият ключов процес е **разглеждане, филтриране и избор на екипировка от клиента**. След вход в системата клиентът може да избере курорт и да разгледа наличните гардероби и артикули. Системата извлича информацията от базата данни и я представя в удобен за визуализация формат. Клиентът може да използва различни критерии за филтриране, като тип екипировка, марка, размер или ценови диапазон. При избор на конкретен артикул, потребителят може да отвори детайлен изглед, който съдържа пълната информация за него.

Петият процес е **създаване на резервация (поръчка)**. След като клиентът избере един или повече артикули, той ги добавя към своята количка и задава период на наемане чрез начална и крайна дата. При потвърждение на заявката системата извършва няколко проверки – валидност на датите, наличие на достатъчно количество от избраните артикули и коректност на въведените данни. След това се създава нов запис в таблицата bookings, а избраните артикули се записват в booking_items. Системата автоматично изчислява общата цена на базата на броя дни, количеството и цената на ден за всеки артикул.

Шестият процес е **обработка и проследяване на резервации**. След създаване на поръчката тя получава определен статус, който може да се променя във времето – например „активна“, „потвърдена“, „отменена“ или „приключена“. Клиентът може да преглежда своите поръчки, а собственикът може да следи заявките, свързани с неговите артикули. Това позволява по-добра организация на наличностите и контрол върху използването на екипировката.

Седмият важен процес е **административен контрол и одобрение**. Администраторът има достъп до специализирани функционалности, чрез които може да преглежда новосъздадени гардероби и да решава дали да ги одобри или отхвърли. Това гарантира, че в системата се публикуват само коректни и надеждни обекти.

Освен основните процеси, системата включва и допълнителни поддържащи процеси, като **възстановяване на парола**, при който се генерира код за нулиране и се проверява неговата валидност, както и процеси по валидиране на входните данни и обработка на грешки.



Фиг. 4. Workflow Diagram

3.7 Достъп до функционалностите от страна на потребителя

В ShredShare достъпът до функционалностите е реализиран чрез мобилен интерфейс, базиран на екранно ориентиран модел. Всеки основен процес в приложението е представен като отделен екран, а преминаването между тях се осъществява чрез навигационна система, изградена с помощта на централен navigation graph. Този подход позволява ясно дефиниране на всички възможни маршрути в приложението и гарантира логическа последователност между действията на потребителя.

Началната точка на взаимодействие със системата е екранът за вход или регистрация. Тук потребителят въвежда своите данни и избира роля, ако създава нов акаунт. След успешна автентикация системата анализира ролята на потребителя и автоматично го пренасочва към съответния начален екран. Това поведение е съществено, тъй като различните типове потребители имат различни цели и съответно се нуждаят от различен набор от функционалности.

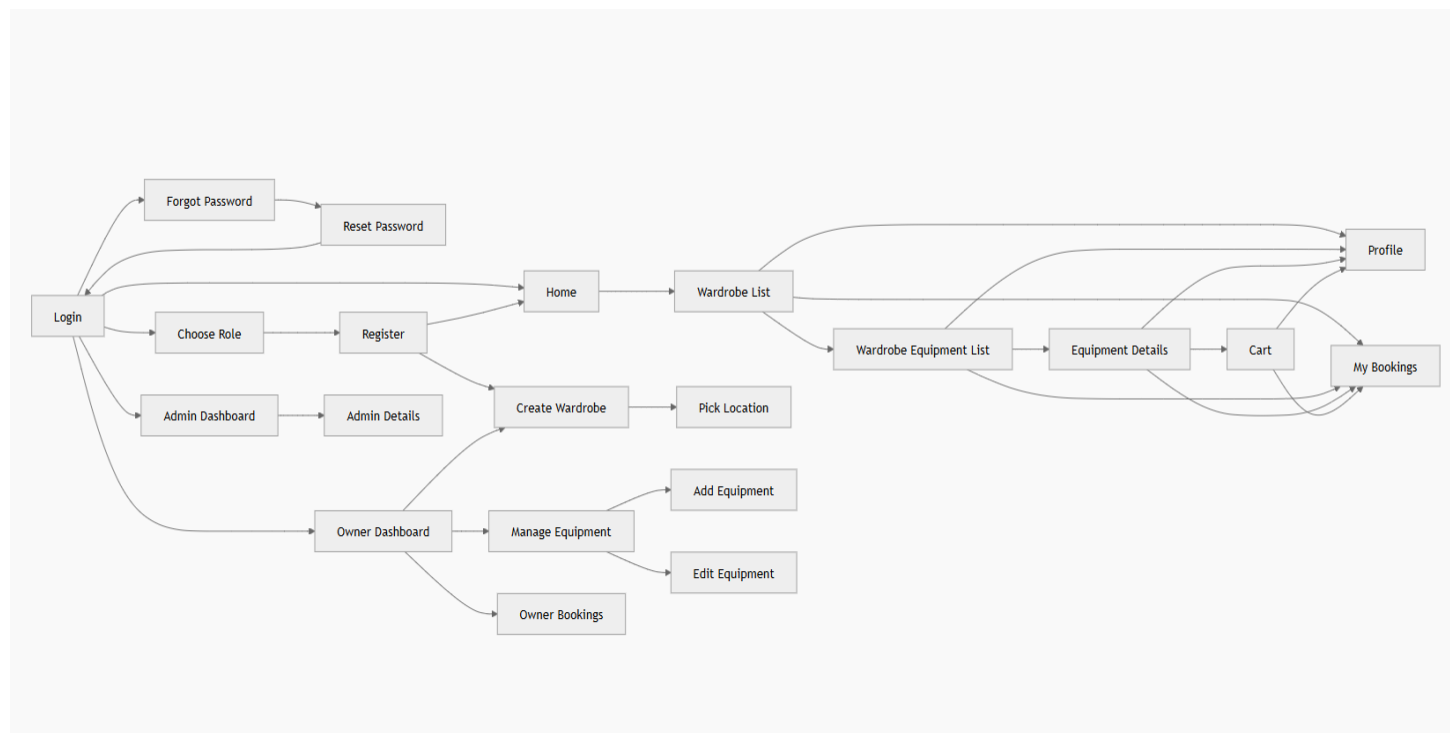
За потребителя с роля **клиент**, основният достъп до функционалности започва от начален екран, който предоставя възможност за избор на курорт и разглеждане на наличната екипировка. От този екран потребителят може да премине към списък с артикули, да използва филтри и сортиране, както и да отвори детайлен изглед на конкретен продукт. От детайлния екран клиентът може да добави артикула към количка и впоследствие да премине към процеса на създаване на резервация. Навигацията е организирана така, че потребителят винаги да може лесно да се върне назад или да премине към следваща стъпка, без да се губи в структурата на приложението.

За потребителя с роля **собственик**, достъпът до функционалности е ориентиран към управление на съдържание. След вход той се насочва към dashboard, който съдържа списък с неговите гардероби и възможност за създаване на нови. Оттам собственикът може да отвори конкретен гардероб, да прегледа наличната екипировка, да добавя нови артикули или да редактира съществуващи. Формите за въвеждане на данни са организирани така, че да предоставят ясна структура и да водят потребителя през необходимите стъпки. Навигацията позволява бърз достъп до всички важни действия, като същевременно избягва излишна сложност.

За потребителя с роля **администратор**, достъпът до функционалности е съсредоточен върху контрол и управление на системата. След вход администраторът получава достъп до специализиран панел, в който може да преглежда заявки за одобрение, да анализира данни за гардероби и потребители и да извършва административни действия. Навигацията в този случай е проектирана така, че да осигури бърз достъп до критичните функции и ясно представяне на информацията, необходима за вземане на решения.

Особено важен аспект при достъпа до функционалностите е използването на **условна навигация**, при която определени екрани и действия са достъпни само при наличие на конкретни условия. Например, потребител без вход не може да създава резервации или да управлява гардероби. По същия начин клиентът няма достъп до административните функции, а собственикът не може да извършва действия, предназначени за администратор. Това осигурява сигурност и правилно функциониране на системата.

Допълнително, в приложението е реализирана и навигация чрез действия, базирани на състоянието на данните. Например, след успешно създаване на поръчка, потребителят се пренасочва към екран с резултат или към списък със своите резервации. При неуспешно действие се визуализира съобщение за грешка и се предоставя възможност за корекция на въведените данни.



Фиг. 5 Навигация между основните екрани

4. Реализация

В този раздел се разглежда практическата реализация на приложението **ShredShare** в съответствие с предварително изготвения проектен модел. Реализацията обхваща всички основни компоненти на системата – база данни, сървърна част, мобилно приложение, бизнес логика, потребителски интерфейс и механизми за валидация и контрол на достъпа. Основната цел е да се покаже как проектираните структури, процеси и връзки са реализирани на практика чрез конкретни технологии и програмен код. Приложението е реализирано като клиент-сървърна система, в която Android приложението изпълнява ролята на клиентска част, Spring Boot приложението реализира сървърната логика и API комуникацията, а MySQL базата данни осигурява постоянно съхранение на информацията. По този начин е постигнато ясно разделение между визуалната част, обработката на заявките и слоя за данни.

4.1 Бази данни

Базата данни на приложението е реализирана чрез **MySQL** и съдържа основните таблици, необходими за работата на системата: потребители, курорти, гардероби, екипировка, резервации и детайли на резервациите. Връзките между таблиците са реализирани чрез външни ключове, което осигурява целостта и последователността на данните.

Една от основните таблици в системата е таблицата users, която съхранява информация за всички типове потребители – клиенти, собственици и администратори:

```
CREATE TABLE users (  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  address VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  phone VARCHAR(20),  
  role ENUM('customer','owner','admin') DEFAULT 'customer'  
);
```

Таблицата resorts съдържа информация за ски курортите, включително местоположение и характеристики. Използван е Dataset, който включва информация за latitude, longitude и останалата информация за улесняване при избор на курорт, като beginner_slopes, intermediate_slopes, child_friendly и др:

```
CREATE TABLE resorts (  
  resort_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  country VARCHAR(100),  
  latitude DECIMAL(9,6) NOT NULL,  
  longitude DECIMAL(9,6) NOT NULL,  
  season varchar(100)  
);  
ALTER TABLE resorts  
ADD COLUMN beginner_slopes INT,  
ADD COLUMN intermediate_slopes INT,  
ADD COLUMN difficult_slopes INT,  
ADD COLUMN total_slopes INT,  
ADD COLUMN child_friendly BOOLEAN,  
ADD COLUMN snowparks BOOLEAN,  
ADD COLUMN gondola BOOLEAN;  
alter table resorts add column highest_point int;
```

За обектите, които отдават екипировка под наем, е реализирана таблицата wardrobes, която е свързана едновременно с потребител и курорт:

```
CREATE TABLE wardrobes (  
  wardrobe_id INT AUTO_INCREMENT PRIMARY KEY,  
  owner_id INT,  
  resort_id INT,  
  name VARCHAR(100) NOT NULL,  
  address VARCHAR(200),
```

```
phone VARCHAR(20),
description TEXT,
latitude DECIMAL(10,7),
longitude DECIMAL(10,7),
```

```
FOREIGN KEY (owner_id) REFERENCES users(user_id),
FOREIGN KEY (resort_id) REFERENCES resorts(resort_id)
```

```
);
```

Екипировката се съхранява в таблицата equipment, където за всеки артикул се пазят данни като тип, марка, размер, цена и количество:

```
CREATE TABLE equipment (
    equipment_id INT AUTO_INCREMENT PRIMARY KEY,
    wardrobe_id INT,
    name VARCHAR(100),
    type VARCHAR(50),
    brand VARCHAR(100),
    size VARCHAR(20),
    price_per_day DECIMAL(10,2),
    quantity INT,
    description TEXT,
```

```
FOREIGN KEY (wardrobe_id) REFERENCES wardrobes(wardrobe_id)
```

```
);
```

Резервациите са реализирани чрез таблиците bookings и booking_items. Първата съдържа общата информация за поръчката, а втората – конкретните артикули в нея:

```
CREATE TABLE bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    start_date DATE,
    end_date DATE,
    total_price DECIMAL(10,2),
    status VARCHAR(50),
```

```
FOREIGN KEY (user_id) REFERENCES users(user_id)
```

```
);
```

```
CREATE TABLE booking_items (
    booking_item_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT,
    equipment_id INT,
```

quantity INT,

FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
FOREIGN KEY (equipment_id) REFERENCES equipment(equipment_id)

);

Така реализираната база данни осигурява надеждна основа за всички основни процеси в приложението – регистрация, управление на гардероби, добавяне на екипировка и създаване на резервации.

4.2 Backend (Spring Boot)

Сървърната част на приложението е реализирана с цел обработка на бизнес логиката и осъществяване на връзката между клиентското приложение и базата данни. Тя приема заявки от мобилното приложение, валидира входните данни, извършва необходимите проверки и връща резултат към клиента.

В сървърната част са организирани основните слоеве на приложението – **контролери, сервизни класове, repository слой и модели за данни**. Контролерите приемат HTTP заявки, сервизният слой изпълнява логиката на приложението, а repository класовете осъществяват връзка с базата данни.

Така например при регистрация или вход в системата заявката се приема от контролер, предава се към съответния service клас, извършва се проверка дали потребителят съществува и при успех се връща резултат към Android приложението. По същия начин се реализират и процесите по добавяне на гардероб, извличане на екипировка, създаване на резервация и административно одобрение.

1. Entry point и конфигурация

Началната точка на бекенд приложението е класът **ShredShareApplication**:

```
package com.shredshare.ShredShare;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ShredShareApplication {

    public static void main(String[] args) {
        SpringApplication.run(ShredShareApplication.class, args);
    }
}
```

Този клас има основна роля, защото чрез него се стартира цялото Spring Boot приложение.

Ключов елемент тук е анотацията **@SpringBootApplication**. Тя е една от най-важните анотации в Spring Boot и комбинира няколко функционалности в една. Чрез нея класът се маркира като главен конфигурационен клас на приложението. Освен това тя активира автоматичната конфигурация на Spring и позволява автоматично откриване на останалите компоненти в проекта, като контролери, услуги, конфигурационни класове и repository интерфейси.

Една от конфигурациите в приложението е SecurityConfig:

```
package com.shredshare.ShredShare.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Този клас отговаря за създаването на bean, който ще се използва за кодиране на потребителските пароли.

@Bean указва, че методът, върху който е поставена анотацията, връща обект, който трябва да бъде регистриран в Spring контейнера. Това означава, че върнатият обект може да бъде използван повторно в други класове. Вместо всеки път ръчно да се създава нов обект, Spring го управлява автоматично.

@Configuration анотацията указва на Spring, че даденият клас съдържа конфигурация и че в него могат да бъдат дефинирани bean-ове, които да се управляват от Spring контейнера. Казано по-просто, тя прави класа част от настройките на приложението

Следващият конфигурационен клас е **SecurityFilterConfig**:

```

package com.shredshare.ShredShare.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityFilterConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(...patterns: "/auth/**").permitAll()
                .anyRequest().permitAll()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

Този клас определя как приложението ще обработва сигурността на входящите HTTP заявки. Най-важният елемент тук е типът **SecurityFilterChain**. Това е механизъм на Spring Security, който описва веригата от правила и филтри, през които преминава всяка входяща заявка. Именно тук се дефинира кои маршрути са достъпни свободно, кои изискват удостоверяване и какъв тип сигурност ще бъде използван.

Методът:

`public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception`

приема параметър от тип **HttpSecurity**. Това е обект, чрез който се настройва поведението на сигурността за HTTP ниво. Чрез него се конфигурират правила като: дали да е активен CSRF, кои пътища са достъпни и др. **CSRF** означава *Cross-Site Request Forgery*. Това е механизъм за защита, използван основно при уеб приложения с форми и браузърни сесии. В случая той е изключен чрез `disable()`, което е често използвано решение при REST API приложения, особено когато клиентът е мобилно приложение, а не стандартен уеб браузър.

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**").permitAll()
    .anyRequest().permitAll()
)

```

Методът **authorizeHttpRequests()** се използва за дефиниране на правилата за достъп до различните HTTP маршрути. Вътре е използван `requestMatchers("/auth/**")`, което означава, че всички адреси, започващи с `/auth/`, ще бъдат разпознати от това правило. Символите `**` означават, че се обхващат всички подмаршрути под този път.

`permitAll()` означава, че достъпът до тези маршрути е разрешен за всички потребители, без значение дали са автентикирани или не. Това е логично за операции като регистрация, вход и забравена парола.

anyRequest() означава „всяка друга заявка“, която не е обхваната от предходните правила. В случая и за тях е зададено permitAll(), което означава, че към момента системата позволява достъп до всички останали маршрути без ограничение.

httpBasic() активира HTTP Basic автентикация — стандартен механизъм, при който потребителско име и парола се изпращат чрез HTTP заявки.

В проекта е реализиран и класът **WebConfig**:

```
package com.shredshare.ShredShare.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/uploads/**")
            .addResourceLocations("file:uploads/");
    }
}
```

Този клас служи за конфигуриране на достъпа до статични файлове, които не се намират в стандартните ресурсни директории на Spring.

Тук класът **implements WebMvcConfigurer** означава, че той имплементира интерфейс, който позволява персонализиране на Spring MVC конфигурацията. Това е начин да се променя стандартното поведение на уеб слоя, без да се подменя цялата му конфигурация.

registry.addResourceHandler("/uploads/**")

Това указва, че всички URL адреси, започващи с /uploads/, трябва да бъдат обработвани като заявки за статични ресурси.

.addResourceLocations("file:uploads/");

определя реалната физическа локация на тези файлове. Префиксът file: означава, че ресурсите се намират в директория от файловата система, а не вътре в classpath ресурсите на приложението.

Глобалните настройки на приложението са зададени във файла **application.properties**:

```
spring.application.name=ShredShare
spring.datasource.url=jdbc:mysql://localhost:3306/shareshred
spring.datasource.username=root
spring.datasource.password=admin
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=shredshareapp@gmail.com
spring.mail.password=nnuqwebykltpske
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000
```

```
server.port=8080
```

Свойството `spring.application.name` задава името на приложението. Свойството `spring.datasource.url` определя адреса на базата данни, към която приложението ще се свързва.

В проекта е реализирана и конфигурация за изпращане на имейли, което е важно за функционалности като **забравена парола** и изпращане на код за възстановяване на достъп. Свойството `spring.mail.host` задава SMTP сървър, през който приложението ще изпраща имейли. В случая е използван SMTP сървърът на Gmail. SMTP е стандартен протокол за изпращане на електронна поща. Свойството `spring.mail.port` определя порта, който ще се използва за връзката със SMTP сървър. Порт 587 е стандартен за защитена комуникация.

Тези две настройки съдържат имейл акаунта, от който се изпращат съобщенията, както и неговата парола или `app password`.

`spring.mail.username` определя кой акаунт ще бъде използван като подател.

`spring.mail.password` съдържа паролата за удостоверяване пред SMTP сървър.

Това позволява на приложението автоматично да изпраща съобщения към потребителите, например при заявка за смяна на парола.

1. Структура

За да бъде бекенд приложението ShredShare добре организирано, лесно за поддръжка и удобно за разширяване, програмният код е структуриран в отделни пакети според тяхното предназначение.

Пакетът **config** съдържа конфигурационните класове на приложението. В него се намират класове, които не реализират директно бизнес логика, а задават основните настройки на системата.

Тук попадат класове като:

- конфигурация за кодиране на пароли;
- конфигурация на сигурността;
- конфигурация на уеб слоя;
- конфигурации за достъп до ресурси и други глобални настройки.

Ролята на този пакет е да централизира всички настройки, които определят как приложението работи, какви bean-ове се създават и как се държи Spring средата при стартиране.

Пакетът **Controller** съдържа контролерите на приложението. Това са класовете, които приемат HTTP заявки от клиентската част и връщат отговори обратно към Android приложението.

Контролерите представляват входната точка към бекенд логиката. Те обработват маршрути, приемат параметри и request body данни, извикват съответните service класове и връщат резултат под формата на JSON или HTTP отговор.

Най-често именно тук се реализират endpoint-и за:

- регистрация и вход;
- зареждане на курорти;
- управление на гардероби;
- добавяне и извличане на екипировка;
- създаване и обработка на резервации;
- административни действия.

Пакетът Controller е важен, защото той осъществява директната комуникация между мобилното приложение и сървърната част.

Пакетът **dto** съдържа така наречените **Data Transfer Objects**. Това са класове, които се използват за пренос на данни между Android приложението и бекенда, както и между отделните слоеве в самото приложение.

Използването на DTO обекти е важно, защото не е добра практика entity класовете да се изпращат директно към клиента. Вместо това се използват специално създадени класове, които съдържат само нужната информация за конкретна операция.

В проекта dto пакетът е допълнително разделен на подпакети според функционалността:

- Admin
- Booking
- Equipment
- Login
- Owner
- Profile
- Register
- Wardrobe

Това разделение е добро архитектурно решение, защото групира DTO класовете според бизнес процесите, в които участват.

Пакетът **Entity** съдържа класовете, които моделират основните обекти в базата данни. Това са така наречените entity класове, които се свързват директно с таблиците в MySQL чрез JPA/Hibernate.

Тук се намират класове като:

- User
- Resort
- Wardrobe
- Equipment
- Booking
- BookingItem

Тези класове съдържат полета, които съответстват на колоните в таблиците, както и анотации за релациите между обектите. Те представляват вътрешния модел на данните в приложението.

Пакетът **Repository** съдържа интерфейсите за достъп до базата данни. Тези класове осъществяват връзката между Java кода и записите в MySQL.

Обикновено repository интерфейсите наследяват Spring Data JPA интерфейси като JpaRepository, чрез което автоматично получават готови методи за:

- търсене;
- запис;
- редакция;
- изтриване;
- извличане по идентификатор.

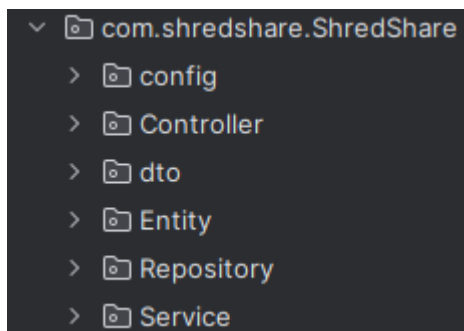
Този пакет има важна роля, защото изолира директната работа с базата данни и позволява на бизнес логиката да използва готов слой за достъп до данните.

Пакетът **Service** съдържа бизнес логиката на приложението. Това е един от най-важните слоеве в цялата архитектура. Именно тук се реализира какво трябва да се случи при дадено действие на потребителя. Service класовете приемат данни от controller слоя, обработват ги, използват repository интерфейсите за достъп до базата данни, прилагат бизнес правила и връщат резултат.

Например в този пакет се реализират операции като:

- регистрация на потребител;
- проверка на входни данни;
- добавяне на гардероб;
- одобрение на гардероб;
- създаване на екипировка;
- изчисляване на цена на резервация;
- обработка на статуси.

Този слой е поставен между controller и repository и служи като основното място за бизнес логиката.



Фиг. 6 Пакетна структура

2. Entity класове

Entity слойът в приложението **ShredShare** представлява връзката между Java кода и базата данни. Той е реализиран чрез **JPA (Jakarta Persistence API)** и се използва за моделиране на таблиците от базата като Java класове. Всеки entity клас съответства на конкретна таблица, а неговите полета – на колоните в нея.

Пример:

```

package com.shredshare.ShredShare.Entity;

import jakarta.persistence.*;
import lombok.Data;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Data
@Entity
@Table(name = "bookings")
public class Booking {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "booking_id")
    private Integer bookingId;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "start_date", nullable = false)
    private LocalDate startDate;

    @Column(name = "end_date", nullable = false)
    private LocalDate endDate;

    @Column(name = "total_price", nullable = false, precision = 10, scale = 2)
    private BigDecimal totalPrice;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 50)
    private BookingStatus status;

    @OneToMany(mappedBy = "booking", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<BookingItem> items;
}

```

Основни анотации:

`@Entity` маркира класа като JPA entity. Това означава, че този клас ще бъде свързан с таблица в базата данни и ще бъде управляван от Hibernate.

`@Table` указва името на таблицата в базата данни, с която е свързан класът. Това е полезно, когато името на класа не съвпада с името на таблицата.

`@Id` маркира полето като първичен ключ. Това означава, че стойността му уникално идентифицира всеки запис в таблицата.

`@GeneratedValue.IDENTITY` означава, че стойността на първичния ключ се генерира от самата база данни (AUTO_INCREMENT в MySQL).

`@Column` се използва за настройка на колоната:

- `name` – име на колоната в базата;
- `nullable` – дали може да бъде NULL;
- `precision` и `scale` – използват се при числови стойности (например `BigDecimal`).

@ManyToOne означава връзка „много към едно“. Това означава, че много записи от текущата таблица могат да сочат към един запис от друга таблица.

@JoinColumn указва коя колона в базата се използва за тази връзка. В случая user_id е външен ключ към таблицата users.

@OneToMany означава връзка „едно към много“. Един запис може да има списък от други записи.

@Enumerated(EnumType.STRING) указва как да се записва enum стойност в базата. STRING означава, че ще се записва като текст (например "PENDING"), а не като число.

@Data е анотация от Lombok, която автоматично генерира getters, setters, toString() и др.

Класът **Booking** представя таблицата bookings и описва една резервация. Останалите Entity класове са създадени по аналогичен начин, спрямо базата данни в MySQL и съответните изисквания.

Entity слойът в ShredShare използва JPA анотации като @Entity, @Table, @Id, @ManyToOne, @OneToMany и @Enumerated, за да свърже Java класовете с базата данни. Чрез тях се дефинират:

- структурата на таблиците;
- връзките между обектите;
- ограниченията върху данните.

Използването на Lombok чрез @Data опростява кода, а enum класовете осигуряват контрол върху допустимите стойности.

3. Repository класове

Repository слойът в приложението ShredShare отговаря за достъпа до базата данни. Той представлява междинен слой между entity класовете и service логиката и позволява работа с данните без ръчно писане на SQL за всяка операция. В проекта този слой е реализиран чрез Spring Data JPA, което значително улеснява извличането, записването, редактирането и изтриването на данни.

Пример:

```

package com.shredshare.ShredShare.Repository;

import com.shredshare.ShredShare.Entity.Booking;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface BookingRepository extends JpaRepository<Booking, Integer> { 3 usages
    List<Booking> findByUser_IdOrderByBookingIdDesc(Integer userId); 1 usage

    @Query(""" 1 usage
        SELECT DISTINCT b
        FROM Booking b
        JOIN b.items bi
        JOIN bi.equipment e
        JOIN e.wardrobe w
        WHERE w.owner.id = :ownerId
        ORDER BY b.bookingId DESC
        """)
    List<Booking> findBookingsByOwnerId(Integer ownerId);
}

```

Основният елемент във всички repository класове е наследяването от интерфейса:

`JpaRepository<EntityType, IdType>`

JpaRepository е част от Spring Data JPA и предоставя готови методи за основни операции върху базата данни, като:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`
- `count()`

Това означава, че дори без допълнително написан код, всеки repository интерфейс автоматично получава базова функционалност.

Всеки repository в проекта е дефиниран като **интерфейс**, а не като клас. Това е важно, защото Spring автоматично създава реализацията му по време на изпълнение. Разработчикът само описва какви методи са нужни, а Spring се грижи да ги реализира.

BookingRepository е по-богато, защото освен стандартните операции съдържа и специализирани методи за извличане на резервации.

`List<Booking> findByUser_IdOrderByBookingIdDesc(Integer userId);`

Този метод използва една от най-силните възможности на Spring Data JPA — **генериране на заявки по име на метода**. Името на метода не е случайно. Spring го анализира и автоматично изгражда SQL/JPA заявка на базата на неговата структура.

Разбивката е следната:

- `findBy` — указва, че се търсят записи;
- `User_Id` — филтрирането става по id на свързания user обект;
- `OrderByBookingIdDesc` — резултатите се сортират по bookingId в низходящ ред.

Това означава, че методът ще върне всички резервации на даден потребител, като най-новите ще са първи.

@Query е анотация, която позволява ръчно писане на заявки към базата данни, които са прекалено многокомпонентни, за да бъдат автоматично генерирани от JpaRepository.

4. DTO класовете

DTO (Data Transfer Object) слой в приложението **ShredShare** се използва за пренос на данни между различните части на системата, най-вече между бекенда и клиентското (Android) приложение. DTO класовете не са директно свързани с базата данни, а служат като междинен формат за обмен на информация.

Основната идея на този слой е да се избегне директното използване на entity класовете при комуникация с клиента. Това позволява:

- по-добър контрол върху това какви данни се изпращат;
- скриване на чувствителна информация (например пароли);
- обединяване на данни от няколко entity обекта в един отговор;
- по-гъвкаво оформяне на JSON отговорите.

Пример:

```
package com.shredshare.ShredShare.dto.Admin;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data 5 usages
@AllArgsConstructor
@NoArgsConstructor
public class AdminDetailsResponse {

    private Integer wardrobeId;
    private String wardrobeName;
    private String resortName;
    private String phone;
    private String address;
    private String description;
    private String imageUrl;
    private String status;

    private Integer ownerId;
    private String ownerFirstName;
    private String ownerLastName;
    private String ownerEmail;
    private String ownerPhone;
    private String ownerRole;
}
```

Класът AdminDetailsResponse е DTO, предназначен за **административната част на приложението**. Той се използва за връщане на подробна информация към администратора относно даден гардероб и неговия собственик.

Този DTO обединява информация от **два различни entity класа**:

- Wardrobe
- User (собственикът)

Това е една от основните причини да се използват DTO обекти — да се съберат данни от различни източници в един структуриран отговор.

Името AdminDetailsResponse показва, че това е **response DTO**, т.е. използва се за връщане на данни към клиента.

Обикновено DTO класовете се делят на:

- **Request DTO** – за входящи данни;
- **Response DTO** – за изходящи данни.

Основни анотации:

@Data

@AllArgsConstructor генерира конструктор с всички полета на класа като параметри.

@NoArgsConstructor генерира празен (default) конструктор без параметри.

Останалите DTO класове са създадени по сходен начин в зависимост от необходимите входящи и изходящи данни.

5. Service класове

Service слой в приложението **ShredShare** съдържа основната бизнес логика на системата. Той стои между controller слоя и repository слоя и има ролята да обработва входните данни, да извършва проверки, да прилага бизнес правила и да използва repository интерфейсите за работа с базата данни. Използването на отделен service слой е важно, защото позволява:

- контролерите да останат по-леки и фокусирани върху HTTP комуникацията;
- бизнес логиката да бъде централизирана;
- кодът да бъде по-лесен за поддръжка и разширяване;
- различни операции да бъдат използвани повторно от други части на системата.

В показаната реализация service слой е реализиран чрез няколко отделни класа, като всеки от тях отговаря за конкретна функционална област.

Всички service класове използват анотацията @Service. Тази анотация указва на Spring, че съответният клас е service компонент и трябва да бъде регистриран в Spring контейнера. Това позволява той да бъде автоматично създаван и инжектиран в други класове, например в controller-и.

AuthService

Класът AuthService отговаря за логиката, свързана с:

- вход в системата;
- регистрация;
- забравена парола;
- смяна на парола чрез код.

EmailService е отделен service клас, който има конкретна отговорност — изпращане на имейли.

```

@Service 2 usages
public class EmailService {

    private final JavaMailSender mailSender; 2 usages

    public EmailService(JavaMailSender mailSender) { no usages
        this.mailSender = mailSender;
    }

    public void sendResetCodeEmail(String toEmail, String code) { 1 usage
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(toEmail);
        message.setSubject("ShredShare - Код за възстановяване на парола");
        message.setText(
            "Здравейте,\n\n" +
            "Вашият код за възстановяване на парола е: " + code + "\n\n" +
            "Кодът е валиден 10 минути.\n\n" +
            "Ако не сте поискали смяна на парола, игнорирайте този имейл.\n\n" +
            "ShredShare"
        );

        mailSender.send(message);
    }
}

```

JavaMailSender е Spring интерфейс, който се използва за изпращане на имейли.
SimpleMailMessage е клас за създаване на обикновен текстов имейл.

BookingService

BookingService съдържа логиката, свързана с:

- създаване на резервации;
- извличане на резервации;
- отказване на резервации;
- промяна на статус от собственик.

EquipmentService

EquipmentService съдържа логиката за:

- създаване на екипировка;
- извличане на екипировка;
- редактиране;
- изтриване;
- качване на снимки.

ResortService

ResortService е най-семплият service клас. Той съдържа базови операции за курортите:

- getAllResorts()
- getResortById()
- saveResort()
- deleteResort()

Тук почти няма бизнес логика — service класът основно препраща към repository слой. Това е подходящо, когато за дадена функционалност не са необходими сложни проверки.

WardrobeService

WardrobeService е един от най-богатите service класове и управлява:

- заявките за гардероби;
- одобрение и отказ;
- статистики;
- списъци за owner и admin;
- създаване на нов гардероб.

UserService

UserService отговаря за потребителския профил.

- getUserProfile(Integer userId)
- updateUserProfile(Integer userId, UpdateProfileRequest request)

Service слойът в ShredShare съдържа същинската бизнес логика на приложението. Чрез анотацията @Service, конструкторното инжектиране и използването на repository интерфейси, той реализира операции като:

- регистрация и вход;
- възстановяване на парола чрез имейл;
- създаване и управление на гардероби;
- качване на изображения;
- добавяне и редакция на екипировка;
- създаване, отказ и управление на резервации;
- управление на профил;
- административни справки и одобрения.

6. Controller класове

Controller слойът в приложението ShredShare представлява входната точка към бекенд логиката. Именно тук се приемат HTTP заявките, изпратени от Android приложението, извличат се подадените параметри, извикват се съответните service методи и се връщат отговори към клиента.

Основната роля на controller класовете е да свържат външната комуникация с вътрешната бизнес логика на системата. Те не съдържат основната обработка на данните, а я делегират към service слоя.

Основни анотации:

@RestController маркира класа като REST контролер. Това означава, че методите в него ще обработват HTTP заявки и ще връщат данни директно като HTTP отговор, обикновено във формат JSON.

@RequestMapping задава основния URL път за целия контролер. Всички методи в този клас ще бъдат достъпни под този префикс. Например @RequestMapping("/auth") или @RequestMapping("/bookings").

@RequestBody указва, че данните идват от тялото на HTTP заявката и трябва да бъдат преобразувани в Java обект. Използва се най-често при JSON заявки, например login, register, create booking, reset password.

@RequestParam извлича параметър от query string или form-data заявка. Използва се при качване на файлове, подаване на стойности през multipart, извличане на параметри от урл заявка.

`@PathVariable` се използва за извличане на стойност от URL пътя. Например при маршрут `/admin/wardrobes/{id}`, стойността на `{id}` ще бъде подадена в параметъра на метода.

`ResponseEntity` представлява HTTP отговор, който може да съдържа:

- тяло на отговора;
- HTTP статус;
- headers.

`ok()` връща HTTP статус 200 и подадения обект като тяло на отговора.

Това е добър подход, защото прави отговорите по-структурирани и позволява по-гъвкав контрол върху HTTP комуникацията.

Тези анотации задават HTTP метода, с който се извиква даден endpoint.

- `@GetMapping` – за извличане на данни;
- `@PostMapping` – за създаване на нов ресурс;
- `@PutMapping` – за редакция или промяна на състояние;
- `@DeleteMapping` – за изтриване.

Пример:

```

@RestController no usages
@RequestMapping("/wardrobes")
@CrossOrigin(origins = "**")
public class WardrobeController {

    private final WardrobeService wardrobeService; 4 usages

    public WardrobeController(WardrobeService wardrobeService) { this.wardrobeService = wardrobeService; }

    @PostMapping(value = "/create", consumes = MediaType.MULTIPART_FORM_DATA_VALUE) no usages
    public ResponseEntity<CreateWardrobeResponse> createWardrobe(
        @RequestParam("ownerId") Integer ownerId,
        @RequestParam("resortId") Integer resortId,
        @RequestParam("name") String name,
        @RequestParam("address") String address,
        @RequestParam("phone") String phone,
        @RequestParam("description") String description,
        @RequestParam("latitude") Double latitude,
        @RequestParam("longitude") Double longitude,
        @RequestParam(value = "imageUrl", required = false) MultipartFile imageUrl
    ) {
        CreateWardrobeResponse response = wardrobeService.createWardrobe(
            ownerId, resortId, name, address, phone, description, latitude, longitude, imageUrl
        );
        return ResponseEntity.ok(response);
    }

    @GetMapping("/owner/{ownerId}") no usages
    public ResponseEntity<List<OwnerWardrobeDto>> getWardrobesByOwner(@PathVariable Integer ownerId) {
        return ResponseEntity.ok(wardrobeService.getWardrobesByOwner(ownerId));
    }

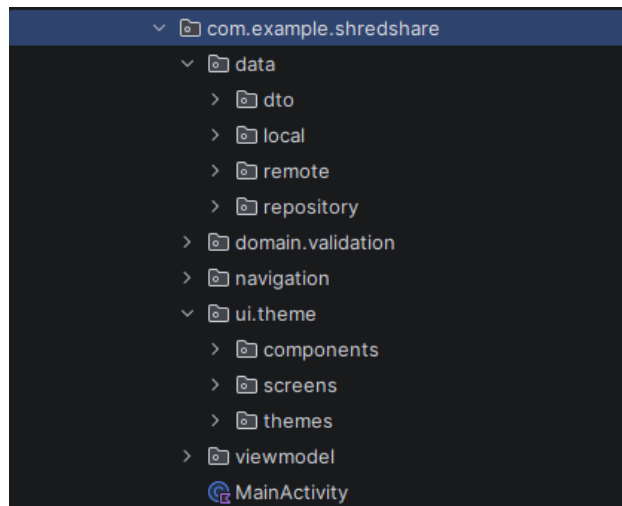
    @GetMapping("/resort/{resortId}") no usages
    public ResponseEntity<List<WardrobeListEquipmentDto>> getApprovedWardrobesByResort(
        @PathVariable Integer resortId
    ) {
        return ResponseEntity.ok(wardrobeService.getApprovedWardrobesByResort(resortId));
    }
}

```

4.3 Android реализация

1. Структура

Android частта на ShredShare е реализирана чрез разделяне на кода на отделни слоеве, което съответства на MVVM архитектурата. Този модел разделя приложението на три основни части: **Model**, **View** и **ViewModel**. Така се постига по-добра организация на кода, по-лесна поддръжка и по-ясно разграничаване между визуализацията, логиката и данните.



Пакет data (Model)

Пакетът data съдържа класовете, свързани с данните и комуникацията с бекенда. Той е разделен на няколко подпакета: dto, local, remote, repository.

dto съдържа DTO класовете, които описват структурата на данните, изпращани към бекенда и получавани от него. Те се използват при login, register, booking, equipment, wardrobe и други заявки.

local съдържа класове за локално съхранение на информация в приложението. Обикновено тук се намира логика като SessionManager, който пази данни за текущия потребител, например userId, role или login status.

remote съдържа Retrofit API интерфейсите. Тези класове описват HTTP заявките към бекенда. Например AuthApi обслужва вход и регистрация, BookingApi — резервациите, EquipmentApi — екипировката, а WardrobeApi — гардеробите. RetrofitInstance е централният клас, чрез който се създава Retrofit клиентът и се задава базовият адрес на сървъра.

repository съдържа repository класовете в Android приложението. Те служат като междинен слой между ViewModel-ите и API интерфейсите. Вместо ViewModel директно да извиква Retrofit, тя използва repository, което прави кода по-чист и по-лесен за промяна.

Пакет validation

Пакетът domain.validation съдържа класове за валидация на потребителските входни данни: AddressValidator, EmailValidator, NameValidator и д.р. Тези класове проверяват дали въведените стойности са коректни, преди да бъдат изпратени към бекенда. Така екраните остават по-чисти, а проверките могат да се използват повторно.

Пакет navigation

Пакетът navigation съдържа логиката за навигация между екраните в приложението.

Тук се намират:

- navigation graph;
- screen routes;
- маршрути с параметри;

- логика за пренасочване според роля.

Този пакет е ключов, защото Android приложението има множество екрани — login, register, client home, owner dashboard, admin dashboard, equipment details, profile, bookings и други. Чрез navigation слоя се определя как потребителят преминава между тях.

Пакет ui (View)

Пакетът ui.theme съдържа визуалната част на приложението. В него се намират екраните, компонентите и темата на приложението.

Тук попадат:

- Compose screens;
- общи UI компоненти;
- цветове;
- стилове;
- бутони, карти, header-и, menu компоненти.

Този пакет е важен, защото ShredShare е реализирано с модерен Android UI подход, при който интерфейсът се изгражда чрез composable функции.

Пакет viewmodel (ViewModel)

Пакетът viewmodel съдържа ViewModel класовете на приложението.

ViewModel-ите имат задачата да:

- пазят състоянието на екрана;
- извикват repository слой;
- обработват резултати от API заявки;
- подават данни към UI;
- управляват loading/error/success състояния.

Това е важен слой при Android архитектурата, защото отделя логиката от визуалната част. UI екраните не трябва директно да изпълняват мрежови заявки, а да използват ViewModel.

В основния пакет се намира **MainActivity**, който е начална точка на Android приложението.

2. Gradle конфигурация и AndroidManifest

Gradle файлът определя основните настройки на Android приложението, използваните технологии, версиите на SDK и външните библиотеки. В проекта **ShredShare** Gradle конфигурацията показва, че приложението е реализирано с **Kotlin**, **Jetpack Compose**, **Retrofit**, **Navigation Compose**, **Google Maps** и **Coil**.

Основните зависимости на gradle са следните:

```
implementation("androidx.navigation:navigation-compose:2.8.0")
```

navigation-compose се използва за навигация между екраните в приложението.

```
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-gson:2.11.0")
```

Retrofit се използва за HTTP комуникация с бекенда.
converter-gson преобразува JSON отговорите към Kotlin обекти и обратно.

```
implementation("io.coil-kt:coil-compose:2.7.0")
```

Coil се използва за зареждане и визуализиране на изображения от URL адреси, например снимки на гардероби и екипировка.

```
implementation("com.google.maps.android:maps-compose:6.1.0")  
implementation("com.google.android.gms:play-services-maps:19.0.0")
```

Тези библиотеки се използват за интеграция на Google Maps в Compose интерфейса.

```
implementation(libs.play.services.location)
```

Използва се за работа с местоположение на устройството.

Файлът AndroidManifest.xml описва основната конфигурация на Android приложението — разрешения, activity класове, достъп до хардуер и външни услуги.

Разрешения:

```
<uses-permission android:name="android.permission.INTERNET" />  
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />  
<uses-permission android:name="android.permission.CAMERA" />
```

Първото разрешение позволява на приложението да използва интернет. Това е задължително, защото ShredShare комуникира с бекенд сървър. Второто и третото разрешение позволяват достъп до местоположението на потребителя. Те са необходими при функционалности, свързани с карти, курорти и позициониране. Последното, позволява използване на камерата, например при качване на снимки за гардероб или екипировка.

```
<meta-data  
    android:name="com.google.android.geo.API_KEY"  
    android:value="@string/google_maps_key" />
```

Тази настройка задава API ключа за Google Maps.

3. MainActivity

Класът MainActivity представлява основната входна точка на Android приложението **ShredShare**. Той е първият компонент, който се стартира при отваряне на приложението и отговаря за инициализирането на потребителския интерфейс.

```

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ShredShareTheme {
                AppNavGraph()
            }
        }
    }
}

```

ComponentActivity е базов клас в Android, който поддържа модерни архитектурни компоненти като Lifecycle, ViewModel и Jetpack Compose. Използването му е стандартна практика при Compose приложения.

Методът onCreate се извиква при стартиране на activity-то. А super.onCreate(...) извиква базовата имплементация и подготвя системата за работа.

Методът setContent е ключов за Jetpack Compose. Той заменя традиционния XML layout подход и позволява директно дефиниране на UI чрез Kotlin код. Всичко, което се намира вътре в setContent, представлява визуалната част на приложението.

AppNavGraph е основният навигационен компонент на приложението. Той определя: всички екрани, маршрутите между тях, логиката за преминаване от един екран към друг.

4. Navigation

Навигацията в Android приложението **ShredShare** е реализирана чрез **Navigation Compose**. Тя определя как потребителят преминава между отделните екрани, как се подават параметри между тях и как се управлява връщането назад или излизането от системата.

Клас Screen

Класът Screen е реализиран като sealed class:

```
sealed class Screen(val route: String) {
    22 Usages
    data object Login : Screen( route = "login")
    2 Usages
    data object ChooseRole : Screen( route = "choose_role")
    3 Usages
    data object Register : Screen( route = "register/{role}") {
        2 Usages
        fun createRoute(role: String): String = "register/$role"
    }
}
```

sealed class позволява всички възможни екрани да бъдат описани на едно място. Това прави навигацията по-подредена и намалява риска от грешки при изписване на route стойности. Всеки екран е дефиниран като data object така вместо навсякъде в кода да се пишат ръчно текстови маршрути като "login" или "cart", се използва: Screen.Login.route, Screen.Cart.route

Някои екрани изискват параметри. Например Register трябва да знае каква е текущата роля. Тук {role} е placeholder в route-а. Методът createRoute() създава реалния маршрут с конкретна стойност. Същият подход се използва навсякъде където е необходимо.

При reset password route-а се използва Uri.encode(email). Това е важно, защото имейл адресът съдържа символи като @ и ., които могат да създадат проблем в URL маршрута и Uri ги преобразува в безопасен формат.

Клас AppNavGraph

AppNavGraph е основната composable функция, която дефинира целия навигационен граф на приложението. Анотацията @Composable показва, че функцията изгражда част от Compose UI.

```
@Composable
fun AppNavGraph() {
    val navController = rememberNavController()
    val context = LocalContext.current
    val sessionManager = remember { SessionManager(context) }
    val cartViewModel: CartViewModel = viewModel()

    NavHost(
        navController = navController,
        startDestination = Screen.Login.route
    ) {
```

В началото се създава nav controller. rememberNavController() създава и запазва навигационния контролер, който управлява преминаването между екраните. Създава се и SessionManager и remember запазва обекта при recomposition, за да не се създава наново при всяко обновяване на UI.

Основата на навигацията е NavHost. NavHost съдържа всички възможни екрани в приложението. startDestination задава началния екран. Тоест приложението започва от login екрана. Всеки екран се дефинира чрез composable маршрути. composable() регистрира конкретен route в навигационния граф и указва кой UI екран да се покаже при навигация към него. При успешен login или register често се използва PopUpTo. Това премахва login екрана от back stack-a, така че след успешен вход потребителят да не може да се върне към него с Back бутона.

За маршрути с параметри се използва:

```
composable(
    route = Screen.OwnerDashboard.route,
    arguments = listOf(navArgument( name = "userId") { type = NavType.IntType })
) { backStackEntry ->
    val userId = backStackEntry.arguments?.getInt( key = "userId") ?: 0
```

Това указва, че route-ът очаква аргумент userId от тип Int и след това стойността се извлича.

В някои случаи се предават допълнителни данни чрез:

navController.currentBackStackEntry?.savedStateHandle?.set(...)

и после се четат чрез:

navController.previousBackStackEntry?.savedStateHandle?.get(...)

Този механизъм се използва например за подаване на:

- latitude;
- longitude;
- име на гардероб.

Това е удобно, когато данните не са част от route-a, но трябва да бъдат достъпни на следващия екран.

В края са дефинирани route composable функции:

```
@Composable
fun OwnerDashboardRoute(
    userId: Int,
    viewModel: OwnerViewModel = viewModel(),
    onCreateWardrobeClick: () -> Unit = {},
    onManageWardrobeClick: (Int) -> Unit = {},
    onManageOrdersClick: () -> Unit = {},
    onLogoutClick: () -> Unit = {}
) {
    val uiState by viewModel.uiState.collectAsState()

    LaunchedEffect( key1 = userId ) {
        viewModel.loadOwnerWardrobes( ownerId = userId )
    }

    OwnerDashboardScreen(
        wardrobes = uiState.wardrobes,
        isLoading = uiState.isLoading,
        error = uiState.error,
        onCreateWardrobeClick = onCreateWardrobeClick,
        onManageWardrobeClick = onManageWardrobeClick,
        onManageOrdersClick = onManageOrdersClick,
        onLogoutClick = onLogoutClick
    )
}
```

Целта на ViewModel-и в navigation route функции е да има междинен слой между навигацията, екрана и данните, вместо самият UI екран директно да зарежда информация.

Тази функция има ролята на **Route слой**. Тя не е самият екран, а подготвя данните за него.

OwnerDashboardRoute прави няколко важни неща:

Първо, създава или взема OwnerViewModel:

```
viewModel: OwnerViewModel = viewModel()
```

Това позволява екранът да използва ViewModel, без самият OwnerDashboardScreen да се занимава със създаване на логика.

Второ, наблюдава състоянието от ViewModel:

```
val uiState by viewModel.uiState.collectAsState()
```

Така когато ViewModel зареди нови данни, например списък с гардероби, UI автоматично се обновява.

Трето, зарежда данните при отваряне на екрана:

```
LaunchedEffect(userId) {  
    viewModel.loadOwnerWardrobes(userId)  
}
```

Това означава, че когато се отвори owner dashboard за конкретен потребител, автоматично се изпраща заявка за неговите гардероби.

Накрая подава готовите данни към UI екрана:

```
OwnerDashboardScreen(  
    wardrobes = uiState.wardrobes,  
    isLoading = uiState.isLoading,  
    error = uiState.error  
)
```

Така OwnerDashboardScreen остава чист визуален компонент. Той само показва данни и бутони, без да знае откъде идват данните.

Същото важи и за:

AdminDashboardRoute

Там route функцията взема AdminViewModel, наблюдава uiState, зарежда dashboard данните чрез:

```
LaunchedEffect(Unit) {  
    viewModel.loadDashboard()  
}
```

и подава резултата към AdminDashboardScreen.

5. Data

Data слой в Android приложението **ShredShare** отговаря за работата с данни. Той включва комуникацията с бекенда, локалното съхранение на информация и repository класовете, които свързват ViewModel слоя с API заявките. Този слой е част от **Model** компонента в MVVM архитектурата.

Remote

съдържа Retrofit конфигурацията и API интерфейсите. Чрез тях Android приложението изпраща HTTP заявки към Spring Boot бекенда. Основният клас е:

```
object RetrofitInstance {  
  
    1 Usage  
    private const val BASE_URL = "http://10.0.2.2:8080/"  
  
    8 Usages  
    private val retrofit: Retrofit by lazy {  
        Retrofit.Builder()  
            .baseUrl(BASE_URL)  
            .addConverterFactory(GsonConverterFactory.create())  
            .build()  
    }  
  
    2 Usages  
    val resortApi: ResortApi by lazy {  
        retrofit.create(ResortApi::class.java)  
    }  
  
    4 Usages  
    val authApi: AuthApi by lazy {  
        retrofit.create(AuthApi::class.java)  
    }  
  
    2 Usages  
    val wardrobeApi: WardrobeApi by lazy {  
        retrofit.create(WardrobeApi::class.java)  
    }  
  
    5 Usages  
    val adminApi: AdminApi by lazy {  
        retrofit.create(AdminApi::class.java)  
    }  
  
    1 Usage  
    val ownerApi: OwnerApi by lazy {  
        retrofit.create(OwnerApi::class.java)  
    }  
  
    5 Usages  
}
```

Той създава централен Retrofit клиент. Адресът 10.0.2.2 се използва при Android Emulator, за да достъпи локалния компютър, на който работи бекендът.

Retrofit.Builder() създава HTTP клиента.

baseUrl() задава основния адрес на бекенда.

GsonConverterFactory.create() позволява автоматично преобразуване между JSON и Kotlin обекти.

build() създава готовия Retrofit обект.

След това за всеки backend controller се създава отделен API интерфейс:

```
val authApi: AuthApi by lazy {  
    retrofit.create(AuthApi::class.java)  
}
```

by lazy означава, че API обектът ще бъде създаден чак когато бъде използван за първи път.

AuthApi съдържа заявките за:

```
login()
register()
forgotPassword()
resetPassword()
```

Той комуникира с /auth endpoint-ите в бекенда и се използва при вход, регистрация и възстановяване на парола.

AdminApi съдържа административните заявки:

```
getPendingWardrobes()
approveWardrobe(id)
rejectWardrobe(id)
getAdminStats()
getWardrobeDetails(id)
```

Тук се използват @GET и @PUT, защото част от заявките само извличат данни, а други променят статус на гардероб.

BookingApi управлява резервациите:

```
createBooking()
getBookingsByUser()
cancelBooking()
getBookingsByOwner()
updateBookingStatusByOwner()
```

При някои заявки се използва @Path. Това означава, че стойността ще бъде поставена в урл адреса.

При отказ на резервация се използва @Query. Това означава, че стойността се изпраща като query параметър.

EquipmentApi и **WardrobeApi** използват @Multipart, защото при създаване или редакция се изпращат не само текстови данни, но и снимка.

Repository

Съдържа repository класовете. Те са междинният слой между ViewModel-ите и Retrofit API интерфейсите. ViewModel-ите не извикват директно API методите, а използват repository класове.

AuthRepository управлява заявките за вход, регистрация и възстановяване на парола.

AdminRepository обслужва административните функции. Той извлича чакащи гардероби, одобрява или отхвърля заявки и зарежда статистики.

BookingRepository съдържа логиката за резервации. Той използва Gson за parsing на грешки, когато backend-ът върне неуспешен отговор. Това е полезно при ситуации като невалидна дата, липса на наличност или неразрешена промяна на статус.

EquipmentRepository управлява създаването, редактирането, извличането и изтриването на екипировка.

WardrobeRepository работи по сходен начин с EquipmentRepository, но за гардероби.

OwnerRepository извлича гардеробите на конкретен собственик.

UserRepository управлява потребителския профил. При редакция на профил, ако backend-ът върне грешка, тя се преобразува чрез Gson към SimpleResponse.

ResortRepository използва Result<List<Resort>>. Този подход е удобен, защото позволява ясно разграничаване между успешен резултат и грешка.

Local

Съдържа класове за локално съхранение на данни чрез SharedPreferences.

```
class SessionManager(context: Context) {  
  
    5 Usages  
    private val prefs: SharedPreferences =  
        context.getSharedPreferences(p0 = "shredshare_session", p1 = Context.MODE_PRIVATE)  
  
    2 Usages  
    fun saveLogin(userId: Int, role: String) {  
        prefs.edit()  
            .putBoolean(p0 = "is_logged_in", p1 = true)  
            .putInt(p0 = "user_id", p1 = userId)  
            .putString(p0 = "role", p1 = role)  
            .apply()  
    }  
  
    fun isLoggedIn(): Boolean {  
        return prefs.getBoolean(p0 = "is_logged_in", p1 = false)  
    }  
  
    4 Usages  
    fun getUserId(): Int {  
        return prefs.getInt(p0 = "user_id", p1 = -1)  
    }  
  
    fun getRole(): String? {  
        return prefs.getString(p0 = "role", p1 = null)  
    }  
  
    10 Usages  
    fun clearSession() {  
        prefs.edit().clear().apply()  
    }  
}
```

SharedPreferences позволява запазване на малки стойности локално в приложението. Методът saveLogin записва дали потребителя е влязъл, неговото ид, неговата роля.

CartManager управлява локалната количка на потребителя.

```

2 Usages
class CartManager(context: Context) {

    3 Usages
    private val prefs = context.getSharedPreferences(p0 = "shredshare_cart", p1 = Context.MODE_PRIVATE)
    2 Usages
    private val gson = Gson()

    3 Usages
    companion object {
        3 Usages
        private const val CART_KEY = "cart_items"
    }

    5 Usages
    fun getCartItems(): List<CartItemDto> {
        val json = prefs.getString(p0 = CART_KEY, p1 = null) ?: return emptyList()
        val type = object : TypeToken<List<CartItemDto>>() {}.type
        return gson.fromJson(json, typeOfT = type) ?: emptyList()
    }

```

Тъй като количката съдържа списък от обекти, тя се записва като JSON чрез Gson:

```
gson.toJson(items)
```

При четене обратно JSON-ът се преобразува до списък:

```
val type = object : TypeToken<List<CartItemDto>>() {}.type
gson.fromJson(json, type)
```

Методите в CartManager са:

```
getCartItems()
addItem(item)
increaseQuantity(equipmentId)
decreaseQuantity(equipmentId)
removeItem(equipmentId)
clearCart()
```

Така количката може да се запази локално, дори ако потребителят временно напусне екрана.

DTO класове

В приложението ShredShare DTO (Data Transfer Object) класовете се намират в пакета data.dto и играят ключова роля в комуникацията между Android приложението и бекенд сървър. Те представляват структури от данни, които описват каква информация се изпраща към сървър и каква се получава обратно. DTO класовете не съдържат бизнес логика и не са обвързани с базата данни – тяхната единствена цел е пренос на данни.

Работата с DTO класовете започва от потребителския интерфейс. Когато потребителят извърши действие, например вход в системата, UI слой създава обект от тип LoginRequest, който съдържа въведените имейл и парола. Този обект се подава към ViewModel-a, който го препраща към repository слой. Repository класът използва Retrofit API интерфейса, за да изпрати заявката към бекенда, като DTO обектът автоматично се преобразува в JSON чрез библиотеката Gson.

След обработка на заявката бекендът връща отговор, който отново е във формат JSON. Retrofit автоматично преобразува този JSON в съответния Response DTO клас, например LoginResponse. Repository класът получава този обект и го връща към ViewModel-a, който от своя страна го подава към

UI слоя за визуализация. По този начин DTO класовете участват във всяка стъпка от комуникацията между клиента и сървъра.

В проекта се използват два основни типа DTO класове. Първият тип са Request DTO класовете, които се използват при изпращане на данни към сървъра. Такива са например LoginRequest, RegisterRequest, CreateBookingRequest, ForgotPasswordRequest и ResetPasswordRequest. Те съдържат само необходимите полета, които backend-ът очаква.

Вторият тип са Response DTO класовете, които описват данните, върнати от сървъра. Примери за такива класове са LoginResponse, RegisterResponse, BookingResponse, UserBookingResponse, EquipmentResponseDto, WardrobeResponse и AdminStatsResponse. Те съдържат информация като статус на операцията, съобщения към потребителя и данни, които трябва да се визуализират.

7. ViewModel

ViewModel слойът в Android приложението **ShredShare** е основна част от MVVM архитектурата. Той стои между потребителския интерфейс и Data слой. Неговата задача е да пази състоянието на екраните, да приема действия от потребителя, да извиква repository класовете и да подава готов резултат към UI слоя. ViewModel класовете не визуализират директно екрани и не съдържат Compose UI. Те управляват данните, зареждането, грешките и успешните операции.

```
2 Usages
class HomeViewModel : ViewModel() {

    1 Usage
    private val repository = ResortRepository(RetrofitInstance.resortApi)

    6 Usages
    private val _uiState = MutableStateFlow( value = HomeUiState())
    val uiState: StateFlow<HomeUiState> = _uiState

    init {
        loadResorts()
    }
}
```

```

3 Usages
class CartViewModel(application: Application) : AndroidViewModel(application) {

    7 Usages
    private val cartManager = CartManager( context = application)
    1 Usage
    private val sessionManager = SessionManager( context = application)
    1 Usage
    private val bookingRepository = BookingRepository()

    3 Usages
    data class CartUiState {
        val items: List<CartItemDto> = emptyList(),
        val startDate: String = "",
        val endDate: String = "",
        val isLoading: Boolean = false,
        val error: String? = null,
        val message: String? = null
    } {
        val isEmpty: Boolean get() = items.isEmpty()
        1 Usage
        val totalPrice: Double get() = items.sumOf { it.pricePerDay * it.quantity }
    }

    20 Usages
    private val _uiState = MutableStateFlow( value = CartUiState())
    val uiState: StateFlow<CartUiState> = _uiState.asStateFlow()

    init {
        loadCart()
    }

```

Основата на всеки ViewModel е наследяването от ViewModel или, когато е необходим достъп до Application context:

```
class CartViewModel(application: Application) : AndroidViewModel(application)
```

AndroidViewModel се използва в случаи като количката, където е необходим достъп до локални мениджъри като CartManager и SessionManager, които използват Context.

За съхранение на състоянието почти всички ViewModel-и използват MutableStateFlow и StateFlow:

```
private val _uiState = MutableStateFlow(AddEquipmentUiState())
val uiState: StateFlow<AddEquipmentUiState> = _uiState.asStateFlow()
```

MutableStateFlow е вътрешното променливо състояние, което може да се редактира само във ViewModel-a.

StateFlow е публичното състояние, което UI слойът наблюдава. Така се спазва принципът, че екранът може да чете състоянието, но не трябва директно да го променя.

Състоянието обикновено се описва чрез data class. Този подход позволява всички данни за даден екран да бъдат събрани на едно място — въведени стойности, loading състояние, грешки, съобщения и резултат от операцията.

Някои ViewModel-и управляват и локално съхранени данни. Например CartViewModel използва CartManager и SessionManager. Това позволява количката и потребителската сесия да се съхраняват локално чрез SharedPreferences. При създаване на резервация ViewModel-ът взема текущия потребител от сесията и артикулите от количката, след което създава заявка към бекенда

В заключение, ViewModel слойът в ShredShare реализира логиката на екраните, без да смесва тази логика с визуалния интерфейс. Той съхранява състоянията чрез StateFlow, изпълнява асинхронни

операции чрез `viewModelScope`, използва `repository` класове за достъп до данни и извършва предварителна валидация чрез отделни `validator` класове.

8. UI

UI слой в Android приложението **ShredShare** е реализиран чрез **Jetpack Compose**. Това означава, че потребителският интерфейс не се изгражда чрез XML файлове, а чрез Kotlin функции, маркирани с анотацията `@Composable`. Анотацията `@Composable` показва, че дадена функция описва част от потребителския интерфейс. В приложението всеки основен екран е реализиран като отделна `composable` функция, например екран за добавяне на екипировка, количка, резервации, карта, детайли за гардероб и управление на артикули.

UI слой не работи директно с базата данни или Retrofit заявките. Той получава готово състояние от `ViewModel` слоя чрез `collectAsState()`:

```
val uiState by viewModel.uiState.collectAsState()
```

Така екранът автоматично се обновява, когато `ViewModel` промени своето състояние. Това е характерно за MVVM архитектурата — UI само визуализира данните и изпраща действия обратно към `ViewModel`-а.

В много екрани е използвано разделение между **Route функция** и **Screen функция**. `Route` функцията свързва екрана с `ViewModel`-а и навигацията, а `Screen` функцията се грижи основно за визуализацията.

```
@Composable
fun AddEquipmentRoute(
    wardrobeId: Int,
    equipmentId: Int? = null,
    viewModel: AddEquipmentViewModel = viewModel(),
    onBackClick: () -> Unit = {},
    onAddSuccess: () -> Unit = {}
) {
```

Тази функция зарежда данните, наблюдава `uiState`, обработва резултати като успешно добавяне и подава нужните стойности към реалния екран.

Основната визуална структура на екраните често е изградена чрез `Box`, `Column`, `Row`, `Card` и `LazyColumn`.

`Box` позволява наслагване на елементи един върху друг. В **ShredShare** това се използва за поставяне на фоново изображение, затемняващ слой и съдържание върху него. Така се постига по-модерен визуален стил.

`Column` подрежда елементите вертикално, а `Row` — хоризонтално. `Card` се използва за визуално отделяне на съдържанието в блокове, например форма за въвеждане, детайли за резервация или информация за гардероб.

Използването на `RoundedCornerShape` и полупрозрачни тъмни цветове създава единен стил в приложението.

LazyColumn е подходящ за списъци, защото зарежда елементите ефективно и е оптимизиран за скролване. В приложението се използва за резервации, артикули, списъци с гардероби и други повтарящи се данни.

За показване на изображения от backend-а се използва библиотеката Coil. AsyncImage зарежда изображение от URL адрес асинхронно, без да блокира интерфейса. Това е използвано при снимки на гардероби и екипировка.

```
Column(modifier = Modifier.padding(all = 20.dp)) {
    if (!wardrobe.imageUrl.isNullOrEmpty()) {
        AsyncImage(
            model = "http://10.0.2.2:8080${wardrobe.imageUrl}",
            contentDescription = wardrobe.name,
            modifier = Modifier
                .fillMaxWidth()
                .height(220.dp)
                .clip(shape = RoundedCornerShape(size = 18.dp)),
            contentScale = ContentScale.Fit
        )

        Spacer(modifier = Modifier.height(18.dp))
    }
}
```

В приложението е реализирана и работа с **Google Maps**. Например в началния екран се показва карта с маркери за ски курортите:

```
GoogleMap(
    modifier = Modifier.fillMaxSize(),
    cameraPositionState = cameraPositionState,
    properties = mapProperties,
    uiSettings = mapUiSettings
) {
    uiState.resorts.forEach { resort ->
        val markerState = rememberMarkerState(
            key = resort.resortId.toString(),
            position = LatLng(resort.latitude, resort.longitude)
        )

        Marker(
            state = markerState,
            title = resort.name,
            snippet = resort.country ?: "",
            onClick = {
                selectedResort = resort
                false
            }
        )
    }
}
```

GoogleMap визуализира картата, а Marker поставя конкретна точка върху нея. Всеки курорт се показва чрез своите координати latitude и longitude. При избор на курорт камерата се премества към него:

```
cameraPositionState.animate(
    update = CameraUpdateFactory.newLatLngZoom(
        LatLng(resort.latitude, resort.longitude),
        zoom = 12f
    ),
    durationMs = 1000
)
```

Google Maps се използва и при избора на локация за гардероб.

За избор на дати в количката е използван DatePickerDialog:

```
if (showDialog.value) {
    DatePickerDialog(
        onDismissRequest = {
            showDialog.value = false
        },
        confirmButton = {
            TextButton(
                onClick = {
                    val selectedMillis = datePickerState.selectedDateMillis
                    val formattedDate = formatDateToApi(selectedMillis)

                    if (formattedDate.isNotBlank()) {
                        onDateSelected(formattedDate)
                    }

                    showDialog.value = false
                }
            ) {
                Text("OK")
            }
        },
        dismissButton = {
            TextButton(
                onClick = {
                    showDialog.value = false
                }
            ) {
                Text("Отказ")
            }
        }
    ) {
        DatePicker(state = datePickerState)
    }
}
```

В UI слоя са използвани и общи компоненти, например AppScreenHeader, който осигурява еднаква горна лента на различни екрани. Това намалява повторението на код и осигурява по-еднакъв визуален стил в приложението.

Components

В допълнение към основните екрани, в приложението **ShredShare** е реализиран отделен пакет с компоненти (ui.theme.components), който съдържа **преизползваеми UI елементи**. Тяхната основна цел е да се избегне дублиране на код и да се осигури **единен дизайн и поведение** във всички екрани. Тези компоненти са изградени отново чрез @Composable функции и могат да се използват навсякъде в приложението.

Един от основните компоненти в приложението е AppScreenHeader, който реализира **унифицирана горна лента (header)** за различните екрани.

```
@Composable
fun AppScreenHeader(
    title: String,
    onBackClick: (() -> Unit)? = null,
    onProfileClick: () -> Unit = {},
    onCartClick: () -> Unit = {},
    onBookingsClick: () -> Unit = {},
    onLogoutClick: () -> Unit = {},
    showProfile: Boolean = true,
    showCart: Boolean = true,
    showBookings: Boolean = true,
    showLogout: Boolean = true
)
```

Компонентът приема:

- **заглавие на екрана (title)**
- **callback функции** за действия (профил, количка, резервации, logout)
- **флагове** (showProfile, showCart и т.н.), които определят кои опции да се показват

Това позволява един и същ компонент да се използва в различни контексти с различно поведение.

Компонентът използва вътрешно състояние чрез remember:

```
var expanded by remember { mutableStateOf(false) }
```

Това състояние контролира дали менюто е отворено.

Това състояние контролира дали менюто е отворено.

Бутонът за меню:

```
IconButton(onClick = { expanded = true }) {  
    Icon(Icons.Default.MoreVert, contentDescription = "Меню")  
}
```

Менюто се реализира чрез DropdownMenu:

```
DropdownMenu(  
    expanded = expanded,  
    onDismissRequest = { expanded = false }  
)
```

UI слойът на ShredShare е реализиран чрез Jetpack Compose и следва модерен декларативен подход. Екраните са изградени чрез composable функции, които получават състояние от ViewModel слоя и визуализират съответните данни. Използвани са компоненти като Box, Column, Row, Card, LazyColumn, OutlinedTextField, Button, DatePickerDialog, ModalBottomSheet и GoogleMap.

9. Validation класове

В приложението **ShredShare** е реализиран отделен пакет `domain.validation`, който съдържа класове за валидиране на входните данни. Този слой е част от **Domain логиката** и има за цел да гарантира, че потребителят въвежда коректни и допустими стойности още преди те да бъдат изпратени към backend-a.

Всички валидатори връщат резултат от тип `ValidationResult`, който е реализиран като sealed class:

```
sealed class ValidationResult {  
    8 Usages  
    object Success : ValidationResult()  
    data class Error(val errorMessage: String) : ValidationResult()  
}
```

Това позволява:

- ясно разграничение между успешна валидация и грешка;
- връщане на конкретно съобщение при грешка;
- лесна обработка чрез when или проверки за тип.

Всеки валидатор е отделен клас, който съдържа метод `validate(...)`. Той приема стойност и връща `ValidationResult`. Например:

```

class EmailValidator {
    fun validate(email: String): ValidationResult {
        if (email.isBlank()) {
            return ValidationResult.Error( errorMessage = "Имейлът е задължителен")
        }
        if (!Patterns.EMAIL_ADDRESS.matcher( input = email).matches()) {
            return ValidationResult.Error( errorMessage = "Невалиден формат на имейла")
        }
        return ValidationResult.Success
    }
}

```

Тук освен проверка за празна стойност се използва и Android utility клас. Това гарантира, че имейлът е в правилен формат (например user@example.com)

5. Потребителско ръководство

1. Начален екран



При стартиране на приложението потребителят вижда началния екран за вход, който представлява основната точка за достъп до функционалностите на системата. Дизайнът на екрана е изчистен и тематично съобразен със зимни спортове, като във фонов режим е използвано изображение с елементи като ски, каски и лифтове, което създава подходяща визуална атмосфера. В горната централна част на екрана е разположено логото на приложението **ShredShare**, което служи за идентификация на системата.

Основната част от екрана съдържа форма за въвеждане на потребителски данни. Тя включва две полета:

- поле за имейл („Имейл“), в което потребителят въвежда своя електронен адрес;
- поле за парола („Парола“), което е защитено (скрит текст), като е налична и икона за показване/скриване на въведената парола.

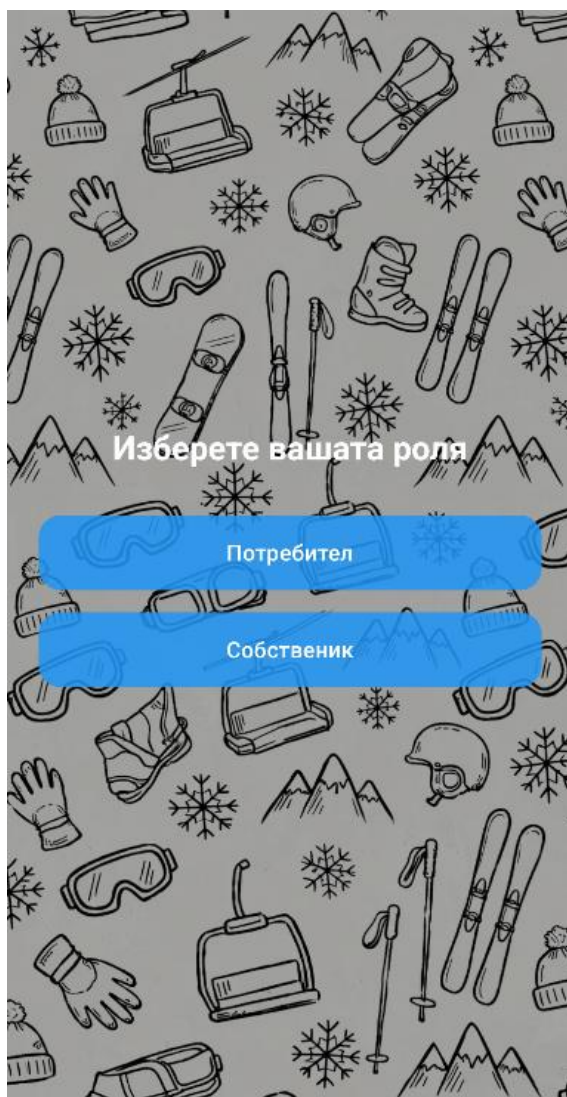
Под полето за парола се намира опция „Забравена парола?“, която позволява на потребителя да премине към процес за възстановяване на достъпа до своя акаунт.

Следва основният бутон „**ВХОД**“, който при натискане изпраща въведените данни за проверка. При коректни данни потребителят се пренасочва към началния екран на приложението.

В случай на грешка (невалиден имейл или парола) системата визуализира съобщение за грешка.

Под бутона за вход е разположен текст „или“, който визуално разделя опциите за вход и регистрация. Най-долу се намира бутонът **„Регистрация“**, чрез който новите потребители могат да създадат акаунт и да получат достъп до функционалностите на приложението.

2. Екран за избор на роля



След натискане на бутона **„Регистрация“**, потребителят бива пренасочен към екран за избор на роля. Този екран е ключов за началната настройка на профила, тъй като определя какви функционалности ще бъдат достъпни в приложението.

В централната част на интерфейса е разположено заглавието **„Изберете вашата роля“**, което ясно насочва потребителя към необходимото действие. Под него са визуализирани два основни бутона:

- **„Потребител“** – изборът на тази роля предоставя достъп до функционалности като разглеждане на наличната екипировка, добавяне на артикули в количката и създаване на резервации.
- **„Собственик“** – тази роля е предназначена за потребители, които желаят да предлагат собствена екипировка под наем чрез създаване и управление на гардероби и артикули.

Изборът на роля е задължителна стъпка, без която потребителят не може да продължи към основната част на приложението. След избора се извършва навигация към съответния интерфейс според избраната роля.

3. Екран за Регистрация

След избора на роля, потребителят бива пренасочен към екрана за регистрация, където трябва да въведе необходимите лични данни за създаване на профил.

В горната част на екрана е разположено заглавието „Регистрация“, както и уточнение за типа регистрация (например „Регистрация като потребител“), което дава ясна индикация за избраната роля.

Формата за регистрация съдържа следните полета за въвеждане на информация:

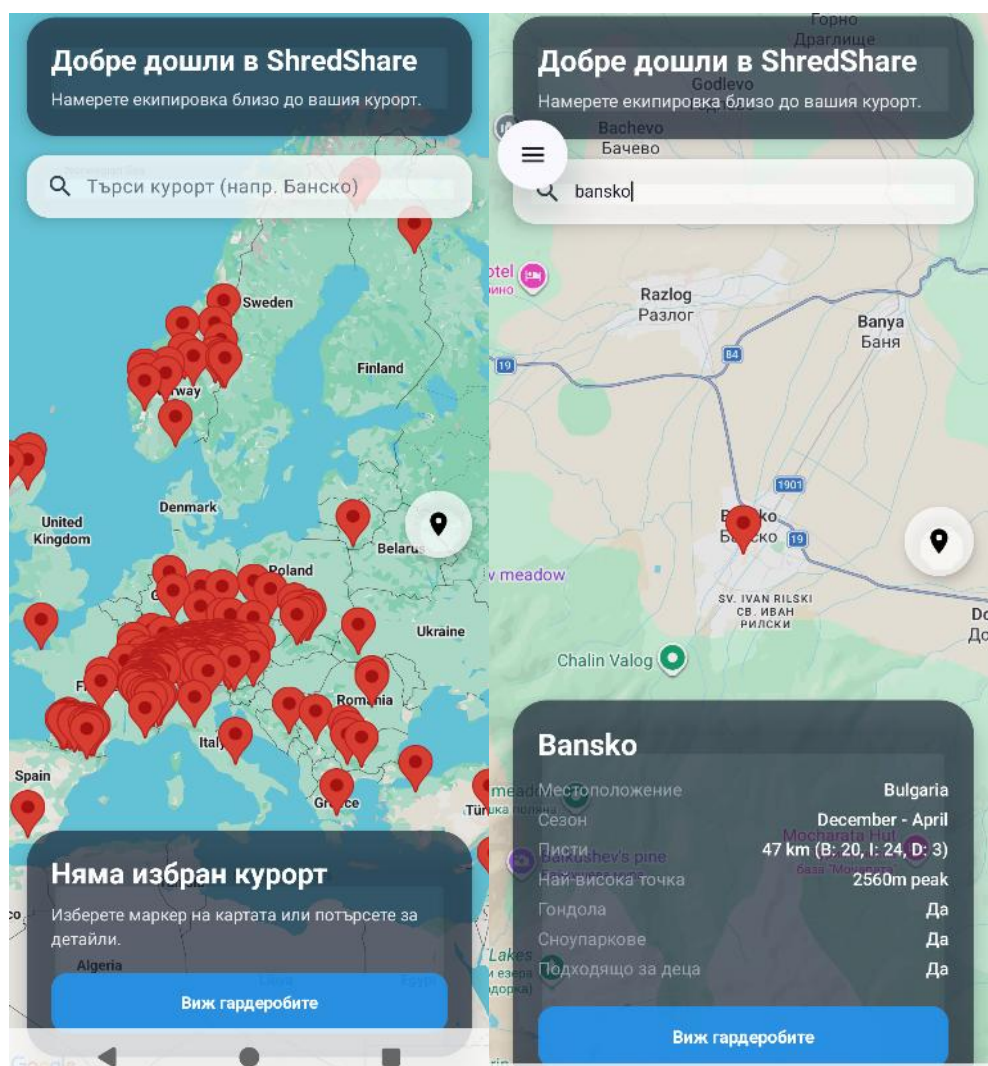
- **Име** – въвеждане на собствено име
- **Фамилия** – въвеждане на фамилно име
- **Адрес** – адрес на потребителя
- **Имейл** – валиден имейл адрес
- **Телефонен номер** – контактен телефон
- **Парола** – създаване на парола (мин-8 символа, малка, главна буква и символ)
- **Потвърди парола** – повторно въвеждане за потвърждение

Всяко поле е придружено от икона, която улеснява визуалното ориентиране и подобрява потребителското изживяване. Дизайнът следва общата стилистика на приложението – полупрозрачен панел върху тематичен зимен фон.

В долната част се намира бутонът „РЕГИСТРАЦИЯ“, чрез който се изпращат въведените данни към системата. Преди създаването на акаунта се извършва автоматична валидация на въведената информация, като при грешки (например невалиден имейл или несъвпадащи пароли) се визуализират съответните съобщения.

При успешно попълване на всички полета и валидни данни, потребителят успешно създава своя профил и получава достъп до функционалностите на приложението според избраната роля.

4. При избор на регистрация като потребител



След успешна регистрация като **потребител**, клиентът автоматично бива пренасочен към началния екран на приложението (**Home Screen**). Това е основният екран, от който започва взаимодействието с функционалностите за търсене и избор на ски курорти.

В горната част на екрана е разположено приветствено съобщение „Добре дошли в ShredShare“, придружено от кратко описание, което насочва потребителя към основната цел на приложението – намиране на екипировка в близост до избран курорт.

Непосредствено под него се намира **поле за търсене**, чрез което потребителят може да въведе името на желан курорт (например „Bansko“). При въвеждане на текст системата подпомага намирането на съответния курорт и визуализира неговата позиция върху картата.

Основната част на екрана представлява **интерактивна карта (Google Maps)**, върху която са отбелязани различни курорти чрез маркери. Всеки маркер съответства на конкретен курорт и може да бъде избран чрез натискане. Картата позволява свободна навигация (приближаване, отдалечаване и преместване), което улеснява откриването на желаната локация.

При избор на конкретен курорт, в долната част на екрана се появява **информационен панел**, съдържащ кратки детайли за него, като например:

- име на курорта
- държава

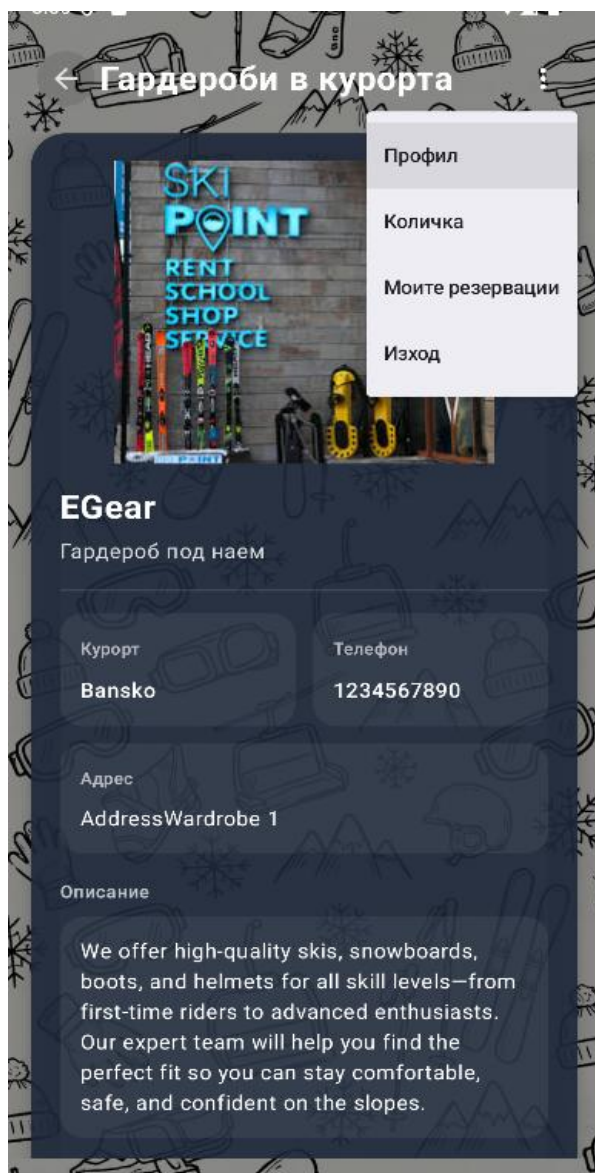
- сезон
- дължина на пистите
- най-висока точка
- наличност на гондола и сноупарк
- дали е подходящ за деца

Този панел предоставя бърз преглед на най-важната информация, без да е необходимо потребителят да напуска екрана.

В долната част на панела е разположен бутон **„Виж гардеробите“**, чрез който потребителят може да премине към следващата стъпка – разглеждане на наличната екипировка под наем за избрания курорт.

Допълнително, в интерфейса е наличен бутон за **текуща локация**, който позволява на потребителя бързо да се позиционира върху картата спрямо своето местоположение.

5. Екран „Гардероби в курорта“



След натискане на бутона **„Виж гардеробите“** от началния екран, потребителят бива пренасочен към екран, който показва всички налични гардероби (пунктове за наем на екипировка) в избрания курорт.

В горната част на екрана е разположено заглавие **„Гардероби в курорта“**, както и бутон за връщане назад. До него е налично и меню (три точки), чрез което потребителят може да достъпва основни функционалности като:

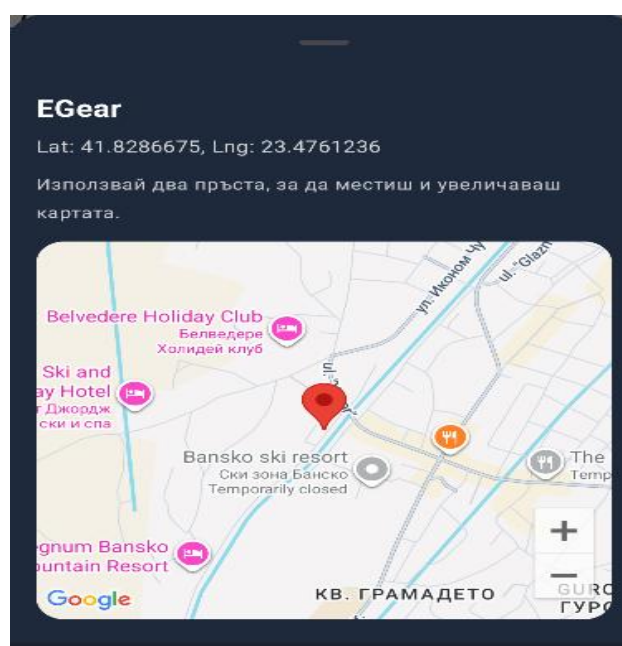
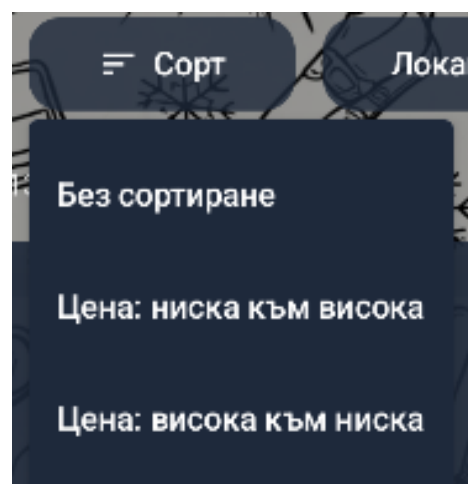
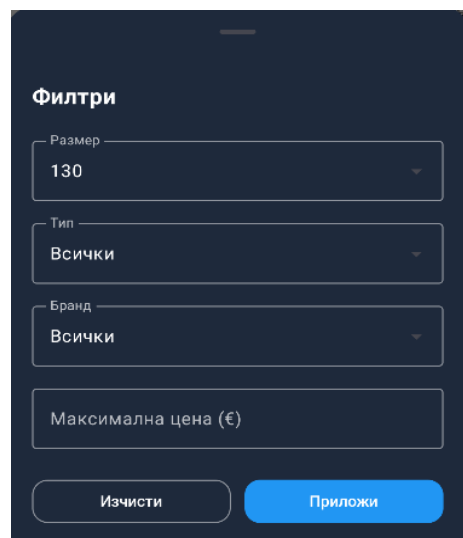
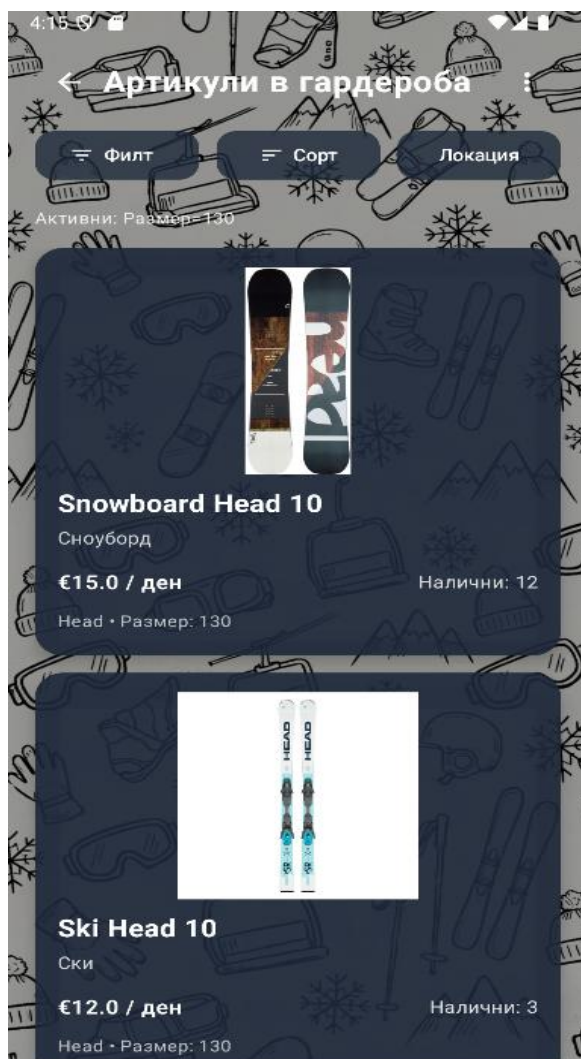
- Профил
- Количка
- Моите резервации
- Изход

Основната част на екрана представя гардеробите под формата на **карти (cards)**, всяка от които съдържа подробна информация за конкретен обект. Всяка карта включва:

- Снимка на гардероба – визуално представяне на обекта
- Име (например „EGear“)
- Кратко описание (например „Гардероб под наем“)
- Курорт, в който се намира (напр. Bansko)
- Телефонен номер за контакт
- Адрес
- Подробно описание на предлаганите услуги и екипировка

Информацията е организирана по ясен и структуриран начин, което позволява на потребителя лесно да сравнява различни гардероби и да избере най-подходящия.

6. Екран „Артикули в гардероба“



След като потребителят избере конкретен гардероб и натисне бутона за разглеждане на наличната екипировка, се отваря екранът **„Артикули в гардероба“**. Този екран предоставя детайлна информация за всички налични артикули под наем, както и допълнителни инструменти за търсене и филтриране.

В горната част е разположено заглавие, заедно с три основни бутона:

- **„Филтър“** – позволява прецизиране на резултатите
- **„Сорт“** – подреждане на артикулите по цена
- **„Локация“** – показва местоположението на гардероба върху карта

Филтриране на артикули

При избор на бутона **„Филтър“** се отваря панел с опции за стесняване на резултатите. Потребителят може да филтрира по:

- **Размер**
- **Тип** (ски, сноуборд и др.)
- **Бранд**
- **Максимална цена (€)**

Налични са бутона:

- **„Изчисти“** – премахва всички зададени филтри
- **„Приложи“** – прилага избраните критерии

Тази функционалност улеснява бързото намиране на подходяща екипировка според нуждите на потребителя.

Сортиране

Чрез бутона **„Сорт“** потребителят може да подреди артикулите по цена:

- без сортиране
- от ниска към висока цена
- от висока към ниска цена

Визуализация на артикули

Всеки артикул е представен чрез карта, която съдържа:

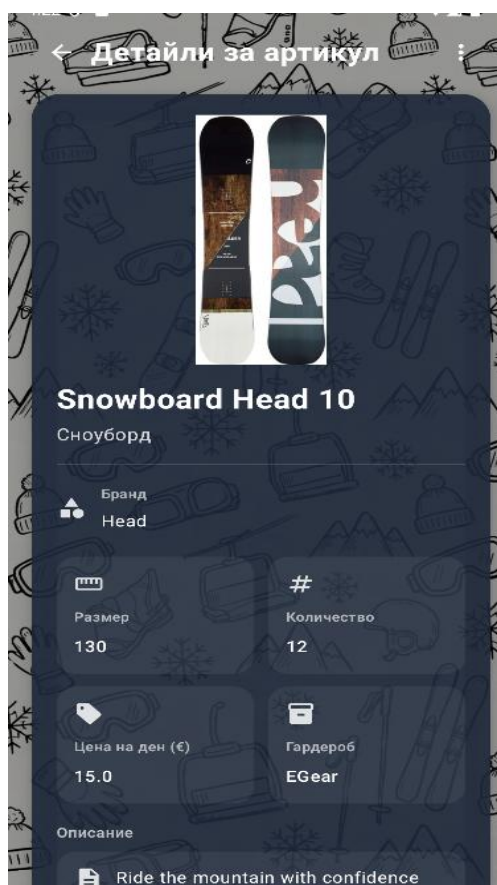
- **Снимка** на артикула
- **Име** (например „Snowboard Head 10“)
- **Тип** (ски, сноуборд и др.)
- **Цена на ден**
- **Наличност (брой)**
- **Допълнителна информация** (например размер)

Информацията е ясно структурирана, което позволява лесно сравнение между различните артикули.

Локация на гардероба

При избор на бутона **„Локация“** се визуализира карта (Google Maps), показваща точното местоположение на гардероба. Потребителят може да използва жестове за приближаване и преместване на картата, което улеснява ориентацията.

7. Екран „Детайли за артикул“



След натискане върху конкретен артикул от списъка, потребителят бива пренасочен към екран с подробна информация за избрания продукт – „Детайли за артикул“.

В горната част на екрана е разположено заглавието, както и бутон за връщане назад. Основното съдържание е организирано в централен панел с изчистен дизайн и ясно структурирана информация.

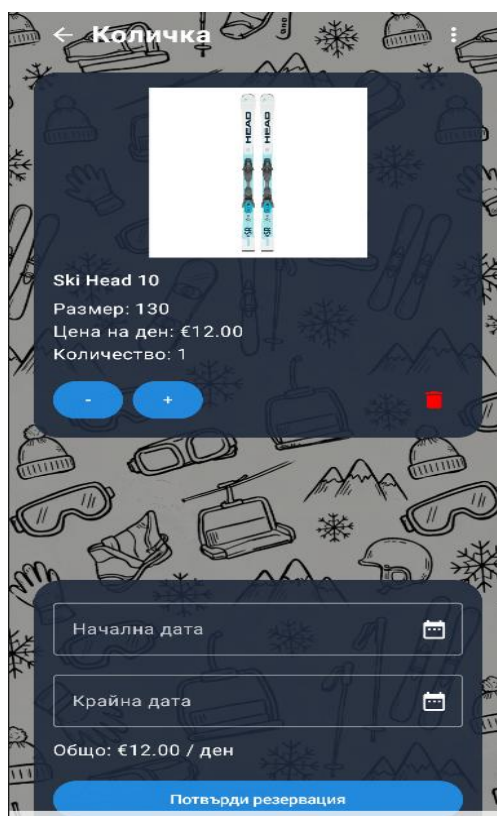
Най-отгоре се визуализира **снимка на артикула**, която дава реална представа за продукта. Под нея са разположени основните характеристики:

- **Име на артикула** (например *Snowboard Head 10*)
- **Тип** (сноуборд, ски и др.)
- **Бранд** (например Head)
- **Размер**
- **Количество (наличност)**
- **Цена на ден (€)**
- **Гардероб**, от който се предлага (например EGear)

Информацията е подредена в отделни секции, което позволява лесно възприемане и бърза ориентация.

В долната част на екрана е налично и **описание на артикула** и бутон „Добави в количка“.

8. Екран „Количка“



Достъпът до екрана „**Количка**“ се осъществява чрез менюто в горната част на приложението (иконата с трите точки), откъдето потребителят избира опцията „**Количка**“. Това позволява бърз достъп до избраните артикули от всяка част на приложението.

Екранът представя всички добавени от потребителя артикули, като всеки от тях е визуализиран в отделна карта със следната информация:

- **Снимка на артикула**
- **Име** (например *Ski Head 10*)
- **Размер**
- **Цена на ден**
- **Количество**

Потребителят разполага с бутони „+“ и „-“, чрез които може да увеличава или намалява количеството на даден артикул. В допълнение, чрез икона за изтриване може напълно да премахне артикула от количката.

В долната част на екрана се намира секция за създаване на резервация, която включва:

- **Начална дата** – избор чрез календар
- **Крайна дата** – избор чрез календар
- **Обща цена на ден** – автоматично изчислявана на база избраните артикули

Системата не позволява избор на минали дати, което гарантира коректност при създаване на резервации.

Най-долу е разположен бутонът „Потвърди резервация“, чрез който потребителят финализира процеса. При успешно създаване на резервация, количката се изчиства автоматично.

9. Екран „Моите резервации“



Достъпът до екрана „Моите резервации“ се осъществява чрез менюто в горната част на приложението, като потребителят избира опцията „Моите резервации“. Този екран предоставя пълен преглед на всички направени резервации.

В горната част е разположено заглавието, както и бутон за връщане назад. Основното съдържание е организирано под формата на карти, като всяка карта представя отделна резервация.

Всяка резервация съдържа следната информация:

- **Номер на резервацията** (например „Резервация #1“)
- **Период** – начална и крайна дата на наема
- **Обща цена**
- **Статус** – показва текущото състояние на резервацията (например *Завършена, Одобрена, Очаква потвърждение, Отказана*)
- **Списък с артикули**, включени в резервацията, заедно с:
 - име на артикула
 - количество
 - цена на ден

Информацията е структурирана ясно и позволява на потребителя бързо да проследи всички свои активности в приложението.

В зависимост от статуса на резервацията, системата може да предоставя допълнителни действия (например възможност за отказ при активни резервации).

10. Екран „Профил“

Екранът „Профил“ се достъпва чрез менюто в горната част на приложението, като потребителят избира опцията „Профил“. Този екран позволява преглед и редакция на личната информация.

В горната част е разположено заглавието „Профил“, както и бутон за връщане назад. Основната част на интерфейса съдържа форма за редакция, обозначена като „Редакция на профил“.

Потребителят има възможност да преглежда и променя следните данни:

- **Име**
- **Фамилия**
- **Адрес**
- **Имейл**
- **Телефонен номер**

Допълнително се визуализира и **ролята на потребителя** (например *customer*), която е информативна и не подлежи на промяна.

Всички полета са предварително попълнени с текущата информация на потребителя, което улеснява редакцията. Интерфейсът следва общия стил на приложението – изчистен дизайн с полупрозрачен панел и добра четимост. В долната част на екрана се намира бутонът „Запази промените“, чрез който потребителят потвърждава направените редакции. При натискане се извършва валидация на въведените данни и при успешно преминаване информацията се обновява в системата.

11. При избор на регистрация като собственик

Създаване на обява за гардероб

Снимка на гардероба

Избери от галерия

Направи снимка

Изберете ски курорт

Изберете или потърсете курорт

Име на гардероба

Въведете име на гардероба

Адрес

Въведете адрес

Телефонен номер

Въведете телефонен номер

Описание

Опишете вашия гардероб...

Местоположение

Все още няма избрано местоположение

Избери на картата

Добави гардероб

Map of Bulgaria showing major cities and a red location pin.

При избор на роля „Собственик“ по време на регистрацията, потребителят получава достъп до функционалност за създаване на собствен гардероб (обява за отдаване на екипировка под наем). След регистрацията той бива насочен към екран за **създаване на гардероб**.

Екранът е структуриран като форма, в която собственикът трябва да въведе необходимата информация за своя обект.

Качване на изображение

В горната част се намира секция за добавяне на снимка:

- „Избери от галерия“ – избор на съществуващо изображение
- „Направи снимка“ – заснемане чрез камерата на устройството

Тази функционалност позволява визуално представяне на гардероба.

Избор на курорт

Следва поле за избор на ски курорт:

- Потребителят може да **избере или потърси курорт** чрез падащо меню
- Възможно е филтриране чрез въвеждане на текст

Основна информация

Формата включва следните задължителни полета:

- **Име на гардероба**
- **Адрес**
- **Телефонен номер**
- **Описание** – допълнителна информация за услугите

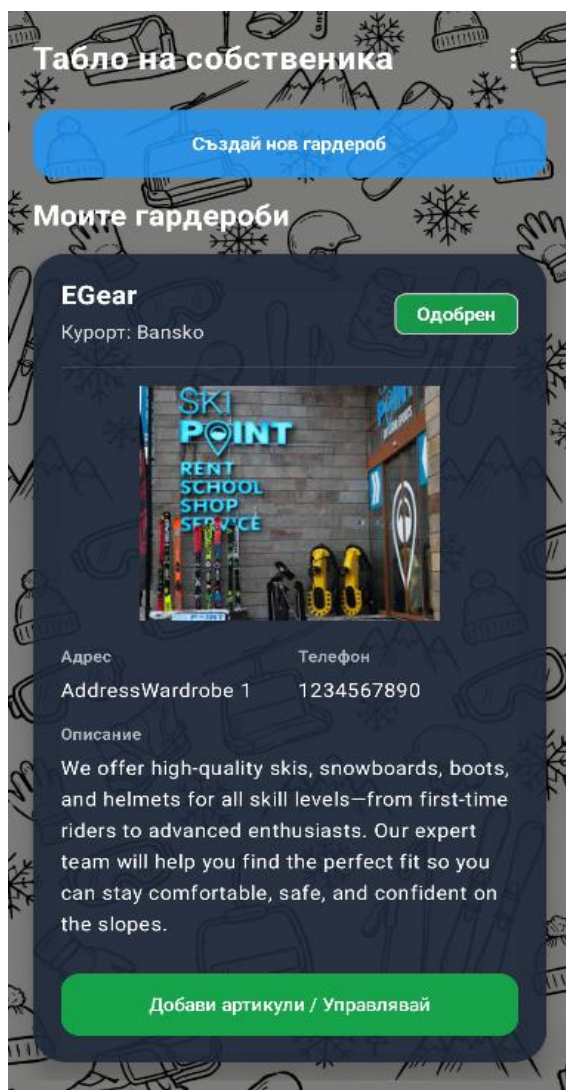
Избор на местоположение

Потребителят трябва да зададе и географска локация:

- Показва се текущо избраната позиция (координати)
- Чрез бутона „Избери на картата“ се отваря карта (Google Maps), където собственикът може да маркира точното местоположение

След избор на точка върху картата, координатите автоматично се попълват във формата. В долната част на екрана се намира бутонът „Добави гардероб“, чрез който се изпращат въведените данни.

12. Екран „Табло на собственика“



След като собственикът създаде заявка за нов гардероб, той бива пренасочен към екрана „Табло на собственика“. Този екран служи като основен контролен панел за управление на създадените гардероби.

В горната част е разположен бутонът „Създай нов гардероб“, който позволява добавяне на нова заявка. Под него се визуализира секцията „Моите гардероби“, където всеки гардероб е представен чрез карта с подробна информация.

Състояние на гардероба

Всеки гардероб има статус, който определя неговата достъпност:

- **„Одобрен“** – гардеробът е прегледан и одобрен от администратор
- **(очаква одобрение / отказан)** – при липса на одобрение

Този статус е ключов, тъй като определя какви действия може да извършва собственикът.

Информация за гардероба

Картата на гардероба съдържа:

- **Име** (например EGear)
- **Курорт**
- **Снимка**
- **Адрес**
- **Телефон**
- **Описание**

Управление на артикули

В долната част на картата се намира бутон:

- „Добави артикули / Управлявай“

Функционалността на този бутон зависи от статуса:

- При **одобрен гардероб** – собственикът може да добавя, редактира и изтрива артикули
- При **неодобрен или отказан гардероб** – тези действия са недостъпни

Процес на одобрение

След създаване на гардероб:

1. Заявката се изпраща към администратор
2. Администраторът я преглежда
3. Взема решение – **одобрение или отказ**

Докато заявката не бъде одобрена, собственикът може единствено да преглежда информацията, без да има достъп до управление на артикули.

13. Екран „Управление на артикули“



След натискане на бутона „Добави артикули / Управлявай“ от таблото на собственика, потребителят бива пренасочен към екрана „Управление на артикули“. Този екран е достъпен само за **одобрен гардероби** и предоставя пълен контрол върху наличната екипировка.

В горната част на интерфейса са разположени основните действия:

- „Добави“ – създаване на нов артикул
- „Филтър“ – прецизиране на списъка с артикули
- „Сорт“ – подреждане на артикулите по зададен критерий

Списък с артикули

Всички артикули са представени чрез карти, като всяка карта съдържа:

- **Снимка на артикула**
- **Име** (например *Snowboard Head 10*)
- **Тип** (сноуборд, ски и др.)
- **Размер**
- **Бранд**
- **Цена на ден (€)**
- **Количество (наличност)**
- **Описание**

Тази информация позволява на собственика да има пълен преглед на всички добавени продукти.

Управление на артикули

В долната част на всяка карта са налични два основни бутона:

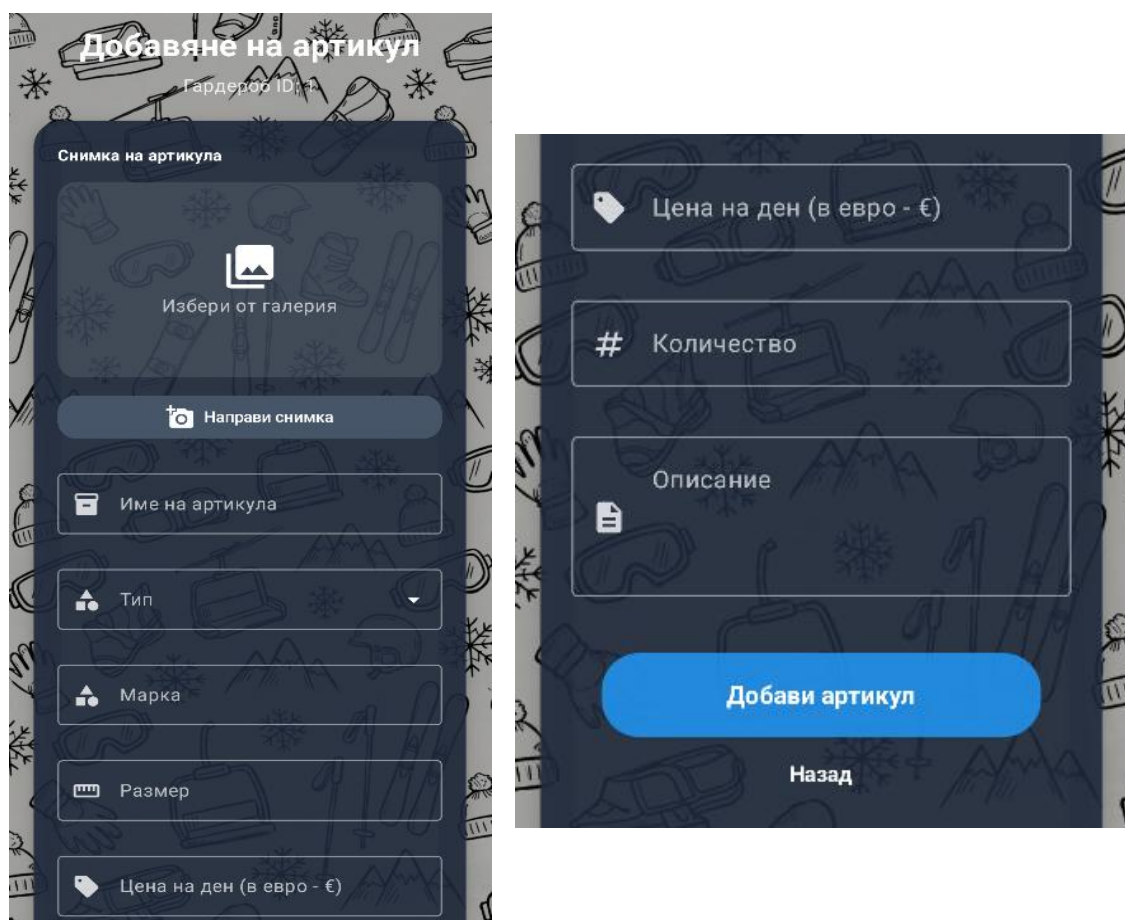
- „**Редактирай**“ – отваря форма за промяна на данните на артикула
- „**Изтрий**“ – премахва артикула от системата (след потвърждение)

Филтриране и сортиране

Подобно на потребителския изглед, собственикът може:

- да филтрира по размер, тип, бранд и цена
- да сортира по цена (възходящо/низходящо)

14. Екран „Добавяне на артикул“



След натискане на бутона „Добави“ в екрана за управление на артикули, собственикът бива пренасочен към екрана „Добавяне на артикул“. Този екран позволява създаването на нов продукт, който ще бъде наличен за наем в съответния гардероб.

В горната част е изписано заглавието, както и идентификаторът на гардероба (**Гардероб ID**), към който ще бъде добавен артикулет.

Качване на изображение

Първата секция е предназначена за добавяне на снимка:

- „**Избери от галерия**“ – избор на съществуващо изображение
- „**Направи снимка**“ – заснемане чрез камерата

Добавянето на изображение не е само визуално, а и важно за по-добро представяне на артикула пред клиентите.

Информация за артикула

Формата включва следните полета:

- **Име на артикула**
- **Тип** – избор от предварително дефинирани категории (ски, сноуборд и др.)
- **Марка (бренд)**
- **Размер**
- **Цена на ден (€)**
- **Количество (наличност)**
- **Описание**

Полето за тип обикновено използва падащо меню, което гарантира коректност и стандартизиране на данните.

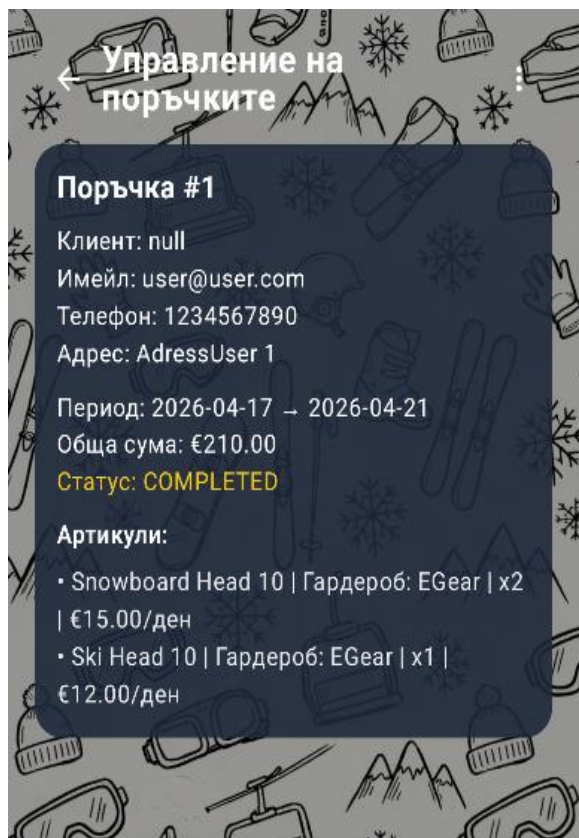
Създаване на артикул

В долната част се намира бутонът „Добави артикул“, чрез който въведената информация се изпраща към системата.

Преди създаването се извършва валидация на всички полета (например дали цената е число, дали количеството е цяло число, дали задължителните полета са попълнени). При наличие на грешки, те се визуализират директно под съответното поле.

Бутонът „Редактирай“ в екран „Управление на артикули“ работи на принципа на този екран като се визуализира последната информация за артикула попълнена от собственика.

15. Екран „Управление на поръчки“



Екранът „Управление на поръчките“ е достъпен чрез менюто в приложението и е предназначен за собствениците на гардероби. Той предоставя възможност за преглед и управление на всички поръчки (резервации), направени от клиентите.

Информация за поръчката

Всяка поръчка е представена чрез карта, съдържаща подробна информация:

- **Номер на поръчката** (например „Поръчка #1“)
- **Клиент** (име, ако е налично)
- **Имейл**
- **Телефон**
- **Адрес**
- **Период** – начална и крайна дата
- **Обща сума (€)**
- **Статус** – текущото състояние на поръчката
- **Списък с артикули**, включващ:
 - име на артикула
 - гардероб
 - количество
 - цена на ден

Тази информация позволява на собственика да има пълен контрол и преглед върху всички активни и завършени резервации.

Управление на статус

Собственикът има възможност да променя статуса на всяка поръчка. Поддържаните състояния са:

- **PENDING** – очаква потвърждение
- **APPROVED** – одобрена поръчка
- **REJECTED** – отказана поръчка
- **PICKED_UP** – екипировката е взета
- **COMPLETED** – поръчката е успешно завършена
- **CANCELLED** – отменена поръчка

Промяната на статуса позволява проследяване на целия жизнен цикъл на една резервация – от създаване до приключване.

16. Екран за администратор



Екранът „Админ панел“ предоставя централизирано място за управление на системата от страна на администратора. Той е основен контролен панел, чрез който се следят и обработват заявките за създаване на нови гардероби, както и обща статистика за потребителите в приложението.

В горната част на екрана са визуализирани основни показатели, които дават бърз преглед на състоянието на системата:

- **Чакащи** – брой заявки за гардероби, които очакват одобрение
- **Одобрени** – брой вече одобрени гардероби
- **Собственици** – общ брой регистрирани собственици
- **Потребители** – общ брой клиенти в системата

Тези стойности позволяват на администратора бързо да оцени натоварването и активността в платформата.

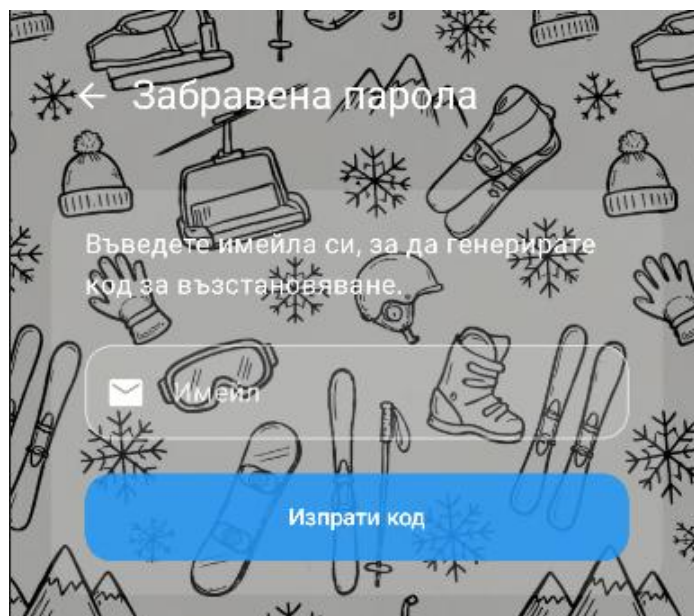
В долната част на екрана се намира секцията „Заявки за гардероби“, където се визуализират всички подадени заявки от собственици за създаване на нов гардероб.

При наличие на заявки, администраторът може:

- да **прегледа детайлна информация** за всеки гардероб
- да **одобри заявката**, което активира гардероба и позволява на собственика да добавя артикули
- да **откаже заявката**, при което гардеробът не се активира

В показания пример няма чакащи заявки („Няма чакащи заявки.“), което означава, че всички подадени заявки вече са обработени.

17. Екран за забравена парола



Екранът „Забравена парола“ предоставя възможност на потребителите да възстановят достъпа до своя акаунт в случай, че са забравили паролата си.

На този екран потребителят трябва да въведе своя **имейл адрес**, който е използван при регистрацията в системата. След това чрез бутона „Изпрати код“ се стартира процес по възстановяване.

Важно условие е:

- въведеният имейл **да бъде вече регистриран и съществуващ в системата**

Ако имейлът е валиден, системата:

- генерира **код за възстановяване**
- изпраща този код към въведения имейл адрес

Формата включва базова валидация:

- проверка дали полето не е празно
- проверка за валиден имейл формат
- проверка дали имейлът съществува в базата данни

При невалиден или несъществуващ имейл се показва съответно съобщение за грешка.

След натискане на бутона „Изпрати код“ и валиден имейл, потребителят бива препратен към екран за въвеждане на получения чрез имейл код възможност за смяна на паролата.

6. Заключение

Разработеното приложение **ShredShare** представлява цялостна система за управление и наемане на ски и сноуборд екипировка, която обединява функционалности за различни типове потребители – клиенти, собственици и администратор. В процеса на разработка е реализирана модерна архитектура, включваща Android приложение с Jetpack Compose за потребителския интерфейс, backend система със Spring Boot и релационна база данни MySQL, което осигурява стабилност, гъвкавост и възможност за бъдещо разширяване.

Приложението може да се счита за завършено и готово за употреба, тъй като покрива всички основни сценарии, необходими за реална експлоатация. Потребителите имат възможност да се регистрират и

вписват, да разглеждат ски курорти чрез интерактивна карта, да получават информация за наличните гардероби и предлаганата екипировка, както и да създават резервации. От своя страна, собствениците могат да създават гардероби, да добавят и управляват артикули, а администраторът разполага с контролен панел за одобрение или отказ на заявки и общо наблюдение на системата.

От гледна точка на потребителските нужди, приложението успешно решава проблема с намирането на подходяща екипировка в близост до избран курорт. Интеграцията с Google Maps, възможностите за филтриране и сортиране, както и подробната информация за всеки артикул допринасят за по-добро потребителско изживяване и улесняват процеса на избор.

По отношение на конкурентоспособността, ShredShare предлага съвременен интерфейс, ясно структурирана логика и разделение на ролите, което го прави сравнимо със съществуващите решения. Въпреки това, съществуват възможности за допълнително развитие, като внедряване на онлайн плащания, система за оценки и коментари, известия в реално време и оптимизация за работа в продукционна среда.

Github Repository: <https://github.com/tppetrov255/ShredShare>